

Shorthand Universal Cycles for Permutations

Alexander E. Holroyd · Frank Ruskey ·
Aaron Williams

Received: 17 September 2010 / Accepted: 16 June 2011 / Published online: 13 July 2011
© Springer Science+Business Media, LLC 2011

Abstract The set of permutations of $\langle n \rangle = \{1, \dots, n\}$ in one-line notation is $\Pi(n)$. The shorthand encoding of $a_1 \cdots a_n \in \Pi(n)$ is $a_1 \cdots a_{n-1}$. A shorthand universal cycle for permutations (SP-cycle) is a circular string of length $n!$ whose substrings of length $n-1$ are the shorthand encodings of $\Pi(n)$. When an SP-cycle is decoded, the order of $\Pi(n)$ is a Gray code in which successive permutations differ by the prefix-rotation $\sigma_i = (1\ 2 \cdots i)$ for $i \in \{n-1, n\}$. Thus, SP-cycles can be represented by $n!$ bits. We investigate SP-cycles with maximum and minimum ‘weight’ (number of σ_{n-1} s in the Gray code). An SP-cycle $\mathbf{nanb} \cdots \mathbf{nz}$ is ‘periodic’ if its ‘sub-permutations’ $\mathbf{a}, \mathbf{b}, \dots, \mathbf{z}$ equal $\Pi(n-1)$. We prove that periodic min-weight SP-cycles correspond to spanning trees of the $(n-1)$ -permutohedron. We provide two constructions: $\mathbf{B}(n)$ and $\mathbf{C}(n)$. In $\mathbf{B}(n)$ the spanning trees use ‘half-hunts’ from bell-ringing, and in $\mathbf{C}(n)$ the sub-permutations use cool-lex order by Williams (SODA, 987–996, 2009). Algorithmic results are: (1) memoryless decoding of $\mathbf{B}(n)$ and $\mathbf{C}(n)$, (2) $O((n-1)!)$ -time generation of $\mathbf{B}(n)$ and $\mathbf{C}(n)$ using sub-permutations, (3) loopless generation of $\mathbf{B}(n)$ ’s binary representation n bits at a time, and (4) $O(n + v(n))$ -time ranking of $\mathbf{B}(n)$ ’s permutations where $v(n)$ is the cost of computing a permutation’s inversion vector. Results (1)–(4) improve on those for the previous SP-cycle construction $\mathbf{D}(n)$ by Ruskey and Williams (ACM Trans. Algorithms 6(3):Art. 45, 2010), which we characterize here using ‘recycling’.

Research supported in part by NSERC.

A.E. Holroyd
Microsoft Research, Redmond, WA, USA
e-mail: holroyd@microsoft.com

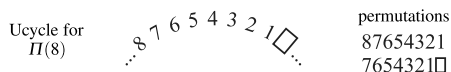
F. Ruskey (✉)
Dept. of Computer Science, University of Victoria, Victoria, Canada
e-mail: ruskey@cs.uvic.ca

A. Williams
Dept. of Mathematics, Carleton University, Ottawa, Canada
e-mail: haron@uvic.ca

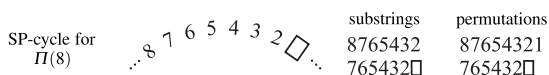
Keywords Ucycles · Gray codes · Cayley graphs · Permutohedron · Algorithms

1 Introduction

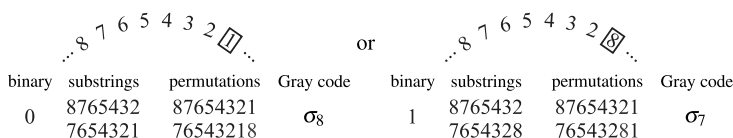
A *universal cycle (Ucycle)* is a circular string containing every object of a particular type exactly once as substring in clockwise order. Ucycles were introduced by Chung, Diaconis, and Graham [1] as generalizations of *de Bruijn cycles*, which are circular strings of length 2^n containing every binary string of length n . A basic set without a Ucycle is $\Pi(n)$, the permutations of $\langle n \rangle := \{1, \dots, n\}$ in one-line notation. For example, a Ucycle for $\Pi(8)$ would contain 87654321 as illustrated below.



Since $7654321\square \in \Pi(8)$, the symbol in the $\square$ must be 8. Similar reasoning fixes each successive symbol, and Ucycles for $\Pi(n)$ only exist when $n \leq 2$. However, Ucycles do exist if we omit the final (redundant) symbol from the one-line notation. The *shorthand encoding* of $a_1 \cdots a_n \in \Pi(n)$ is $a_1 \cdots a_{n-1}$. A *shorthand Ucycle for permutations (SP-cycle)* is a Ucycle that contains the (unique) shorthand encoding of each string in $\Pi(n)$. The *substrings* of an SP-cycle are its $n!$ substrings of length $n-1$. These substrings are the $(n-1)$ -permutations of $\langle n \rangle$, so SP-cycles for $\Pi(n)$ are also Ucycles of the $(n-1)$ -permutations of $\langle n \rangle$. Each substring of an SP-cycle can be *decoded* into a string in $\Pi(n)$ by appending its *missing symbol* from $\langle n \rangle$. These decoded strings are the SP-cycle's *permutations*. For example, an SP-cycle for $\Pi(8)$ must contain the substring 8765432 that is shorthand for its permutation $87654321 \in \Pi(8)$ as illustrated below.



Since $765432\square$ is shorthand for a string in $\Pi(8)$, the $\square$ contains 1 or 8 and the next permutation is 76543218 or 76543281 . These two permutations are obtained by applying σ_8 or σ_7 to the indices of 87654321 , where $\sigma_i := (1 \ 2 \ \cdots \ i)$ is the *prefix-shift* or *prefix-rotation* of length i . Decoding an SP-cycle gives a $(\sigma_n \sigma_{n-1})$ -Gray code since successive permutations in $\Pi(n)$ differ by σ_n or σ_{n-1} . An SP-cycle's *binary representation* has $n!$ bits equaling 0/1 when successive permutations differ by σ_n/σ_{n-1} . The *weight* of an SP-cycle is the number of 1s in its binary representation. The two choices for successive substrings, permutations, prefix-shifts, and bits from the previously illustrated SP-cycle for $\Pi(8)$ appear below.



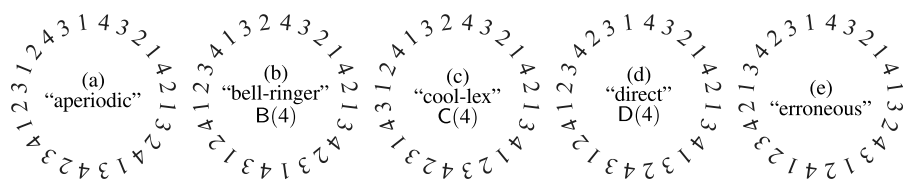


Fig. 1 (a) an aperiodic SP-cycle, (b) the bell-ringer SP-cycle, (c) the cool SP-cycle, (d) the direct SP-cycle, and (e) is not an SP-cycle due to erroneous substring 141 and an extra copy of 342

The *rank* of a substring in a Ucycle is its start position relative to the ‘first’ substring $n-1 \dots 2$ with rank 0. An SP-cycle for $\Pi(n)$ is *periodic* if every n th symbol is n . If $\mathbf{nanb} \dots \mathbf{nz}$ is a periodic SP-cycle for $\Pi(n)$, then $\mathbf{a}, \mathbf{b}, \dots, \mathbf{z}$ are its *sub-permutations* and $\mathbf{na}, \mathbf{nb}, \dots, \mathbf{nz}$ are its *blocks*. The $(n-1)!$ sub-permutations in a periodic SP-cycle equal $\Pi(n-1)$. The substrings, permutations, Gray code, binary representation, sub-permutations, and blocks of a periodic SP-cycle for $\Pi(4)$ are summarized below.

rank	0, 1, 2, 3, 4, ... 19, 20, 21, 22, 23
substrings	432, 321, 214, 142, 421, ... 341, 413, 132, 324, 243
permutations	4321, 3214, 2143, 1423, 4213, ... 3412, 4132, 1324, 3241, 2431
Gray code	$\sigma_4, \sigma_4, \sigma_3, \sigma_3, \sigma_4, \dots \sigma_3, \sigma_4, \sigma_4, \sigma_3, \sigma_3$
binary	0 0 1 1 0 ... 1 0 0 1 1
blocks	4321, 4213, 4231, 4312, 4123, 4132
sub-permutations	321, 213, 231, 312, 123, 132

Figure 1 illustrates the three periodic SP-cycle constructions that are discussed in this article: (b) the *bell-ringer SP-cycle* $B(4)$, (c) the *cool SP-cycle* $C(4)$, and (d) the *direct SP-cycle* $D(4)$ from [13]. It also illustrates (a) an *aperiodic SP-cycle*, and (e) a circular string of length $4!$ over $\langle 4 \rangle$ that is not an SP-cycle for $\Pi(4)$.

1.1 History

Jackson [6] proved that Ucycles exist for the k -permutations of $\langle n \rangle$ when $k < n$. Knuth [9] (p. 75, ex. 111) suggested using $k = n-1$ for encoding permutations, and asked for a construction. Ruskey and Williams [13] answered this request by defining the direct SP-cycle $D(n)$ and generating its symbols in worst-case $O(1)$ -time using $O(n)$ -space. Permutations have also been encoded using relative order [1]. For example, 321341 is an *order-isomorphic Ucycle* since its substrings are *order-isomorphic* to 321, 213, 123, 231, 312, 132. Johnson [7] verified a conjecture [1] by constructing these Ucycles for $\Pi(n)$ using $\langle n+1 \rangle$. Johnson’s Ucycles can also be represented by $n!$ bits, but do not provide simple Gray codes of $\Pi(n)$ when decoded.

Gray codes for permutations and their generation algorithms are well-studied (see Sedgewick [15] or [9]). These algorithms store a single ‘current’ permutation in an array or linked list, and this data structure is modified to create successive permutations. The run-time accounts only for these data structure changes, and not the output of each permutation. In this context, *constant amortized time (CAT)* and *loop-less* algorithms are said to *visit* successive permutations in amortized and worst-case $O(1)$ -time, respectively. An important permutation Gray code is the Johnson-

Trotter-Steinhaus order [8] in which successive permutations differ by an *adjacent-transposition* (or *swap*) from applying $\tau_i := (i \ i+1)$ to the indices of the current permutation. For example, the *JTS-order* for $\Pi(3)$ appears below.

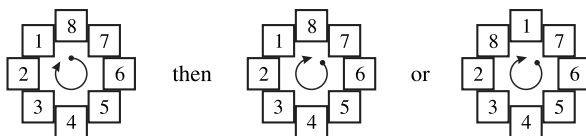
$$123, 132, 312, 321, 231, 213. \quad (1)$$

1.2 Applications

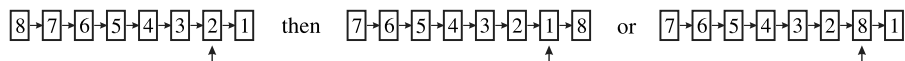
The efficient generation of simple combinatorial objects (including permutations) is fundamental to computer science. For sample applications, see the latest version of Knuth's *The Art of Computer Programming* [10] which devotes 10% of its 3168 pages to “generating all possibilities”. Universal cycles also have applications, with the special case of de Bruijn cycles being used in optical shape acquisition [12], psychology experiments [16], and planar location [14]. This subsection highlights potential applications for SP-cycles. In many cases the weight of an SP-cycle directly influences the application, and this motivates our focus on *minimum-weight* (*min-weight*) and *maximum-weight* (*max-weight*) SP-cycles later in this article.

Efficient Encoding. SP-cycles encode the $n!$ permutations in $\Pi(n)$ using $n!$ symbols over $\langle n \rangle$, and the binary representation reduces this requirement to $n!$ bits.

Efficient Decoding. In a circular array or linked list, σ_n increments the starting position, while σ_{n-1} increments the starting position and swaps the last two symbols. This is illustrated below with arrows depicting the order of symbols in the array.



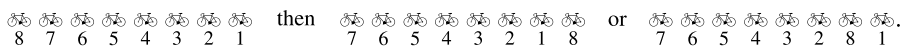
The operations can also be performed in $O(1)$ -time in a singly-linked list, so long as a pointer to the second-last node is maintained.



Suppose an SP-cycle for $\Pi(n)$ contains the consecutive symbols $u_i \cdots u_{i+n-1}$, where indices are taken modulo $n!$. Then the i th permutation can be transformed into the $(i+1)$ st permutation by applying σ_{n-1} if $u_i = u_{i+n-1}$ and by σ_n if $u_i \neq u_{i+n-1}$. Therefore, a given SP-cycle can be decoded by a loopless algorithm when the current permutation is stored in a circular array or linked list. The decoding algorithm is not loopless using a conventional array, since σ_n and σ_{n-1} would both require $\Omega(n)$ -time.

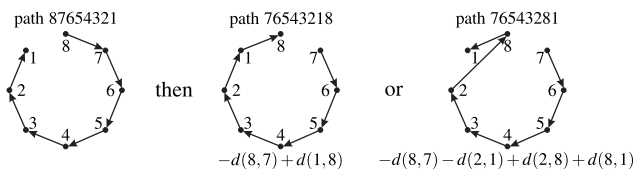
Efficient Operations. In cycling, breakaway groups organize themselves into a tightly-packed single-file line. Riders in the front reduce the wind-resistance for the riders behind, and at regular intervals the lead rider surrenders their position to conserve energy. If the front rider reinserts themselves into the last position (via σ_n)

or second-last position (via σ_{n-1}), then at most one other rider must slow down to accommodate this change. This is illustrated below for riders traveling right-to-left



Proceeding in a $(\sigma_n - \sigma_{n-1})$ -Gray code ensures that riders spend an equal time in each position (equalizing expended energy) and teams have their riders in consecutive positions at the front equally often (equalizing their chance for a further breakaway).

Efficient Evaluation. In some applications the *value* of a permutation depends on its ordered pairs of adjacent symbols. For example, consider the complete directed graph with arc labels $d(i, j)$ giving a distance from node i to node j for $1 \leq i, j \leq n$. The node order in a Hamilton path is represented by $\mathbf{a} = a_1 \cdots a_n \in \Pi(n)$ and its length is $d(\mathbf{a}) = d(a_1, a_2) + d(a_2, a_3) + \cdots + d(a_{n-1}, a_n)$. If σ_n or σ_{n-1} is applied to $\mathbf{a}$, then at most two ordered pairs of adjacent symbols are changed, so the path length changes by at most two additions and two subtractions ($\pm$ update), as seen below.



Therefore, an SP-cycle provides a loopless algorithm for generating and evaluating all $n!$ paths, so long as $d(i, j)$ can be added and subtracted in $O(1)$ -time. This algorithm could be used to determine the distribution of Hamilton path lengths, as is required when judging heuristic or human solutions to the Traveling Salesman Problem (TSP) [11]. The algorithm could also be used when it is feasible (or necessary) to solve the NP-hard problem of determining the shortest possible length (TSP), or the existence of a path with a given length. Similarly, the algorithm could be used to solve generalizations of TSP such as the *stacker crane problem* (see Williams [18]).

Notice that $(\sigma_n - \sigma_{n-1})$ -Gray codes are ‘better’ in these applications than adjacent-transposition Gray codes. Each swap requires two or three $\pm$ updates; τ_1 and τ_{n-1} require two $\pm$ s since they swap the first and last symbol of $\mathbf{a} \in \Pi(n)$, respectively. In JTS-order, the number of successive permutations requiring two $\pm$ updates, t_n , satisfies $t_n \leq 2(n-1)! + t_{n-1}$. Thus, $t_n \leq 2(1! + 2! + \cdots + (n-1)!)$ and so the proportion requiring three $\pm$ updates is asymptotically $(n-2)/n$. We will show that in the Gray codes arising from $B(n)$ and $C(n)$, the proportion requiring one $\pm$ update is asymptotically $(n-2)/n$; this is optimal in the sense that $\lim_{n \rightarrow \infty} (n-2)/n = 1$. In any Gray code, the proportion requiring one $\pm$ update is at most $(n-1)/n$. This is because σ_n and σ_n^{-1} are the operations requiring one $\pm$ update, and $\sigma_n^n = (\sigma_n^{-1})^n = \text{id}$. That proportion was obtained by Compton and Williamson [2] from a Hamilton cycle in the undirected Cayley graph $\text{Cay}(\{\sigma_n, \sigma_2\}, \mathbb{S}_n)$, where $\mathbb{S}_n$ is the symmetric group corresponding to $\Pi(n)$. Their cycle uses arc labels in blocks of $\sigma_2 \sigma_n^{n-1}$ or $\sigma_2 (\sigma_n^{-1})^{n-1}$.

Efficient Ranking. A *ranking algorithm* computes the rank of any substring in a given UCycle. A de Bruijn cycle with a ranking algorithm using $O(n \log n)$ operations exists [9] (p. 25). An oft-cited application of this result is as follows: Suppose a robot wishes to know its position along a closed route. If the route is painted with the 2^n black and white squares of a de Bruijn cycle, then the robot's position is determined by reading n consecutive squares and applying the ranking algorithm (see [14] for a real-world application). In these applications, SP-cycles offer two small advantages (at the expense of using n symbols): (i) each substring contains $n-1$ unique symbols in $\langle n \rangle$ so a single misread color can be detected with probability $(n-2)/(n-1)$, and (ii) fewer squares need to be read to determine the position. In this article, we rank $B(n)$ using $O(n \log n)$ operations. As far as the authors are aware, $B(n)$ is now the second UCycle that can be ranked this quickly. Furthermore, its ranking algorithm appears to be simpler than the ranking algorithm for the aforementioned de Bruijn cycle. A ranking algorithm requiring $\Omega(n^2)$ operations exists for $D(n)$ [13].

1.3 Notation

Strings are given by lowercase bold letters and their symbols are indexed by increasing subscripts. For example, if $\mathbf{a} = 54123$, then $a_1 = 5$ and $a_2 = 4$. Symbols can be concatenated to strings as in $6\mathbf{a} = 654123$. Circular strings are given in uppercase and are indexed circularly. For example, if $\mathbf{U} = 321312$, then $u_7 = u_1 = 3$. Permutations in cycle notation are given by Greek letters, and are multiplied left-to-right. For example, if $\alpha = (1\ 2)$ and $\beta = (1\ 3)$, then $\alpha\beta = (1\ 2\ 3)$. Recall $\sigma_i = (1\ 2\ \dots\ i)$ and $\tau_i = (i\ i+1)$. Exponentiation is repeated multiplication, so $\alpha^2 = \alpha\alpha$. Permutations are applied to the indices of a string written to its left. That is, if $\mathbf{a} = a_1 \dots a_n$ and $\pi = (\pi_1\ \dots\ \pi_n)$, then $\mathbf{a}\pi = a_{\pi_1} \dots a_{\pi_n}$. For example, if $\mathbf{a} = 54123$, then $\mathbf{a}\sigma_3 = 41523$. Brackets fix the order of operations. For example, if $\mathbf{a} = 54123$, then $(6\mathbf{a})\sigma_3 = 654123\sigma_3 = 546123$ and $6(\mathbf{a}\sigma_3) = 6(41523) = 641523$.

1.4 Article Outline

Section 2 investigates max-weight and min-weight SP-cycles. Section 3 constructs min-weight periodic SP-cycles $B(n)$ and $C(n)$. Memoryless rules for decoding $B(n)$ and $C(n)$ are in Sect. 4. CAT algorithms that generate sub-permutations or blocks of $B(n)$ and $C(n)$ are in Sect. 5. Section 6 gives a loopless algorithm that generates blocks of n bits in the binary representation of $B(n)$. Section 7 ranks the permutations in $B(n)$. Algorithms in Sects. 4–7 improve upon those for $D(n)$ [13]. The COCOON 2010 conference included a preliminary version of this article [5].

2 Characterizations

In Sect. 2.1 we show that SP-cycles correspond to certain Eulerian and Hamilton cycles. We bound the weight of an SP-cycle in Sect. 2.2 and discuss those with min-weight and max-weight in Sect. 2.3. Finally, we prove min-weight periodic SP-cycles correspond to spanning trees of the permutohedron by Sect. 2.4.

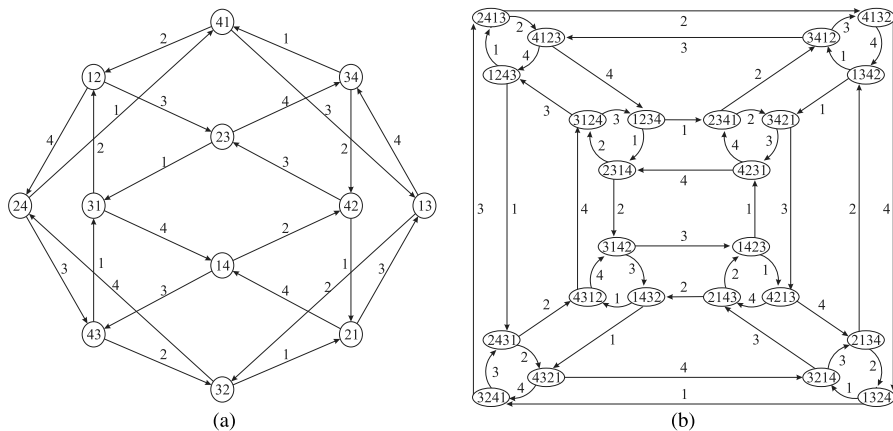


Fig. 2 (a) Jackson graph $J(4)$, and (b) Cayley graph $\Xi(4)$. The straight arcs in $\Xi(4)$ are for σ_4 arcs, and the curved arcs are for σ_3 arcs. The action labels in $\Xi(4)$ are omitted

2.1 The Jackson Graph $J(n)$ and the Cayley Graph $\Xi(n)$

This subsection defines two directed graphs and discusses their relationship to SP-cycles. Nodes in the *Jackson graph* $J(n)$ are the $(n-2)$ -permutations of $\langle n \rangle$, and arcs labeled a_{n-1} are directed from $a_1 a_2 \cdots a_{n-2}$ to $a_2 a_3 \cdots a_{n-1}$ if $a_1 a_2 \cdots a_{n-1}$ is an $(n-1)$ -permutation of $\langle n \rangle$. $J(n)$ is Eulerian since it is strongly connected and the in- and out-degree of each node is two. Figure 2 shows (a) $J(4)$ and (b) $\Xi(4)$ (defined below).

Theorem 1 [6] *SP-cycles for $\Pi(n)$ are in one-to-one correspondence with the arc labels on directed Eulerian cycles in $J(n)$.*

Let $\Xi(n) := \overrightarrow{\text{Cay}}(\{\sigma_n, \sigma_{n-1}\}, \mathbb{S}_n)$ denote the directed Cayley graph on $\Pi(n)$ with generators σ_n and σ_{n-1} . Each arc $(\mathbf{a}, \mathbf{b})$ has *symbol label* a_1 . If $\mathbf{b} = \mathbf{a}\sigma_n$ then $(\mathbf{a}, \mathbf{b})$ is a σ_n arc and has *action label* σ_n ; otherwise $(\mathbf{a}, \mathbf{b})$ is a σ_{n-1} arc with action label σ_{n-1} .

Theorem 2 *SP-cycles for $\Pi(n)$ are in one-to-one correspondence with the symbol labels on directed Hamilton cycles in $\Xi(n)$.*

Proof Notice that $\Xi(n)$ is the directed line graph of $J(n)$; arcs from $a_1 a_2 \cdots a_{n-2}$ to $a_2 a_3 \cdots a_{n-1}$ in $J(n)$ become the unique node $a_1 a_2 \cdots a_n$ in $\Xi(n)$, and each arc in $\Xi(n)$ represents a directed path of length two in $J(n)$. Since directed Eulerian cycles of a directed graph translate into directed Hamiltonian cycles of its directed line graph, then SP-cycles for $\Pi(n)$ are precisely the directed Hamilton cycles in $\Xi(n)$. $\square$

Corollary 1 *SP-cycles for $\Pi(n)$ are in one-to-one correspondence with (σ_n, σ_{n-1}) -Gray codes of $\Pi(n)$.*

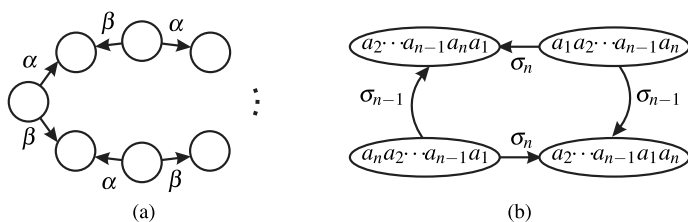


Fig. 3 (a) An alternating α - β cycle in Ξ , and (b) a σ_n - σ_{n-1} cycle in the special case of $\Xi(n)$

Proof Action labels on Hamilton cycles of $\Xi(n)$ are in one-to-one correspondence with $(\sigma_n$ - $\sigma_{n-1})$ -Gray codes of $\Pi(n)$, so this corollary follows from Theorem 2. $\square$

Hamilton paths in $\Xi(n)$ extend to Hamilton cycles in $\Xi(n)$ (Lemma 2.3 [13]), so Corollary 1 implies that Gray codes of $\Pi(n)$ using σ_n and σ_{n-1} are always *cyclic*.

2.2 Maximum and Minimum Weight SP-cycles

In this subsection we provide upper and lower bounds on the weight of an SP-cycle. We discuss an arbitrary finite directed Cayley graph $\Xi := \overrightarrow{\text{Cay}}(\{\alpha, \beta\}, G)$ of a group G with generators α and β , and the special case of $\Xi(n)$ where $\{\alpha, \beta\} = \{\sigma_n, \sigma_{n-1}\}$. Notice that α and β^{-1} (equivalently α^{-1} and β) form alternating cycles in Ξ , as illustrated by Fig. 3(a). We call these alternating cycles α - β cycles. Each node in Ξ belongs to two α - β cycles, and the length of an α - β cycle is twice the order of $\alpha\beta^{-1}$. In the special case of $\Xi(n)$, these σ_n - σ_{n-1} cycles have length four because

$$\sigma_n \sigma_{n-1}^{-1} = \sigma_{n-1} \sigma_n^{-1} = (1 \ n) \quad \text{and} \quad \sigma_{n-1}^{-1} \sigma_n = \sigma_n^{-1} \sigma_{n-1} = (n-1 \ n)$$

have order two. An σ_n - σ_{n-1} cycle in $\Xi(n)$ is illustrated in Fig. 3(b).

A *cycle partition* of Ξ is a set of directed cycles in which each node appears in exactly one directed cycle. Hamilton cycles are cycle partitions with a single cycle.

Remark 1 Given an α - β cycle C , a cycle partition of Ξ either contains all of the α arcs of C and none of its β arcs, or all of the β arcs of C and none of its α arcs.

We use Remark 1 to prove Theorem 3, whose bounds are not necessarily tight.

Theorem 3 Consider the finite directed Cayley graph $\Xi := \overrightarrow{\text{Cay}}(\{\alpha, \beta\}, G)$ of a group G with two generators α and β . Suppose α , β and $\rho := \alpha\beta^{-1}$ have respective orders A , B , and P . In any directed Hamilton cycle, the number N of β arcs is a multiple of P , and satisfies

$$\frac{P}{P-1} \left(\frac{|G|}{A} - 1 \right) \leq N \leq |G| - \frac{P}{P-1} \left(\frac{|G|}{B} - 1 \right).$$

Proof It suffices to prove the first inequality; the second then follows by exchanging α and β . Remark 1 proves the number of β arcs is a multiple of P .

Given any cycle partition, changing all β arcs to α arcs (or vice versa) in one α - β cycle results in a new cycle partition. Furthermore, at most P cycles are altered during each of these changes, so the number of cycles in the cycle partition will increase (or decrease) by at most $P - 1$. If we start from a cycle partition containing a single (Hamilton) cycle, then we may change β arcs to α arcs in α - β cycles until we get to the cycle partition consisting of all α arcs, which has $|G|/A$ cycles. Obtaining the additional $(|G|/A) - 1$ cycles in the cycle partition requires changing at least $((|G|/A) - 1)/(P - 1)$ individual α - β cycles. Each change reduces the β arcs in the cycle partition by P until all are removed. Therefore, the number of β arcs in the initial Hamilton cycle was at least $P \cdot ((|G|/A) - 1)/(P - 1)$ as claimed. $\square$

The following corollary applies Theorem 3 to the special case of $\Xi(n)$, where $|G| = n!$, the generator orders are $\{A, B\} = \{n - 1, n\}$, and $P = 2$.

Corollary 2 *The number of occurrences N of σ_n in a Hamilton cycle of $\Xi(n)$ satisfies*

$$2n \cdot (n - 2)! - 2 \leq N \leq n! - 2(n - 1)! + 2. \quad (2)$$

Similarly, the number of occurrences N of σ_{n-1} in a Hamilton cycle of $\Xi(n)$ satisfies

$$2(n - 1)! - 2 \leq N \leq n \cdot (n - 3) \cdot (n - 2)! + 2. \quad (3)$$

The above inequality also bounds the weight of an SP-cycle for $\Pi(n)$.

2.3 Quotients Graphs $\Xi_n(n)$ and $\Xi_{n-1}(n)$

In this subsection we prove that the upper and lower bounds on the weight of an SP-cycle from Sect. 2.2 can be obtained. The simple connected undirected graph $\Xi_n(n)$ is obtained from $\Xi(n)$ by contracting each directed cycle using n successive σ_n arcs into a single vertex. Two vertices are adjacent in $\Xi_n(n)$ if and only if their cycles have adjacent nodes in $\Xi(n)$. The vertices of $\Xi_n(n)$ represent each coset of $\mathbb{S}_n$ with respect to its subgroup generated by σ_n . Thus, $\Xi_n(n)$ is the *quotient graph* of $\Xi(n)$ where the underlying equivalence relation is string rotation. We label each vertices in $\Xi_n(n)$ with the appropriate string rotation that begins with n . Figure 4 illustrates (a) $\Xi(4)$ (arc labels omitted) and (b) $\Xi_4(4)$ (the octahedron).

Each vertex in $\Xi_n(n)$ has degree n , and each edge in $\Xi_n(n)$ represents exactly two oppositely directed arcs in $\Xi(n)$. In particular, if the label on a vertex in $\Xi_n(n)$ is treated as a circular string, then its neighbors are obtained by each of the n ‘adjacent’-transpositions around the circular string. This is illustrated in Fig. 5 by (a) a directed cycle using eight σ_8 arcs in $\Xi_8(8)$, and (b) its corresponding vertex in $\Xi_8(8)$ with a circular label. To understand why this adjacency rule is true in general, notice that if $\mathbf{a} = a_1 \cdots a_n$ is treated as a circular string, then $\mathbf{a}\sigma_{n-1} = a_2 \cdots a_{n-1}a_1a_n$ gives an ‘adjacent’-transposition between a_1 and a_n when $a_2 \cdots a_{n-1}a_1a_n$ is viewed circularly.

The correspondence given in Theorem 4 is illustrated in Fig. 6.

Theorem 4 *Min-weight SP-cycles of $\Pi(n)$ are in one-to-one correspondence with the spanning trees of $\Xi_n(n)$.*

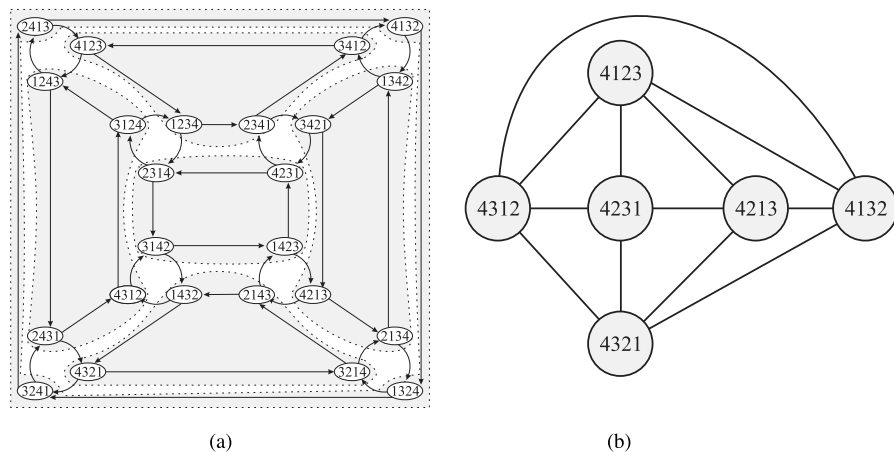


Fig. 4 (a) Cayley graph $\Xi(4)$ with directed cycles using σ_4 arcs highlighted, and (b) the quotient graph $\Xi_4(4)$ obtained by contracting these cycles

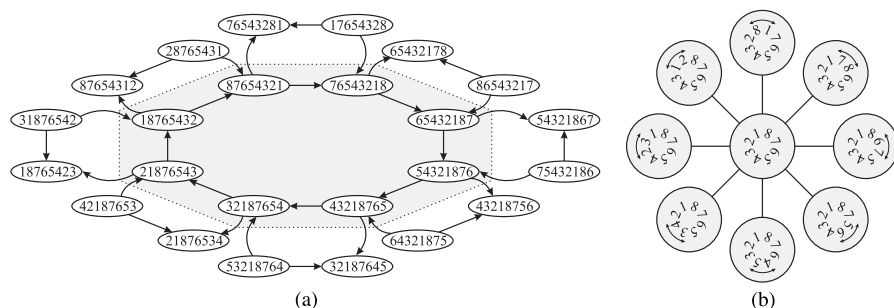


Fig. 5 (a) The directed cycle using σ_8 arcs containing 87654321 in Cayley graph $\Xi(8)$, and (b) the neighborhood of 87654321 in its quotient graph $\Xi_n(n)$. Straight and curved arcs represent σ_8 and σ_7 arcs respectively in (a); vertex labels are written circularly in (b) with arrows highlighting 'adjacent'-transpositions

Proof The number of nodes in $\Xi_n(n)$ is $(n-1)!$, and so a spanning tree of $\Xi_n(n)$ contains $(n-1)!-1$ edges. Given a spanning tree of $\Xi_n(n)$, construct a cycle partition of $\Xi(n)$ in two steps: (1) For each edge in the spanning tree, add the two corresponding σ_{n-1} arcs to the cycle partition, then (2) add σ_n arcs to the cycle partition until each node has out degree 1. This cycle partition is a Hamilton cycle of $\Xi(n)$, and the number of σ_{n-1} arcs in the Hamilton cycle is $2(n-1)!-2$. Therefore, the corresponding SP-cycle is a min-weight SP-cycle by Corollary 2.

Given an SP-cycle of weight $2(n-1)!-2$, its Hamilton cycle in $\Xi(n)$ contains exactly one σ_{n-1} to and from each σ_n coset. Furthermore, each pair of arcs correspond to an edge in $\Xi_n(n)$. These $(n-1)!-1$ edges form a spanning subgraph of $\Xi_n(n)$. Therefore, the min-weight SP-cycle corresponds to a spanning tree of $\Xi_n(n)$. $\square$

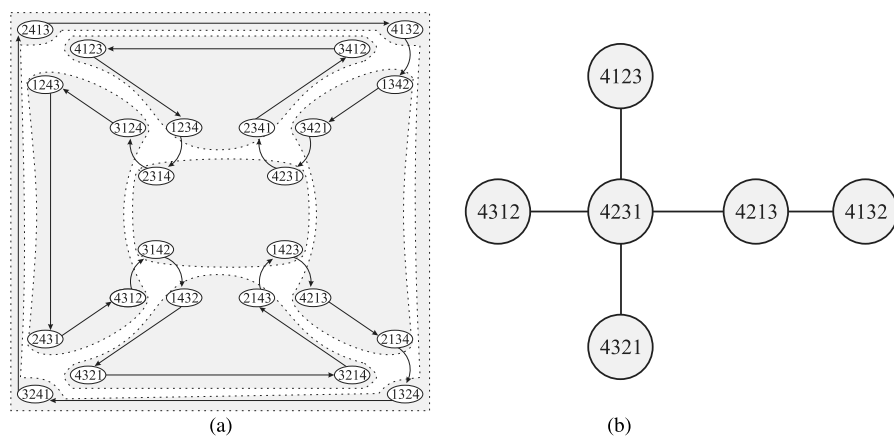


Fig. 6 The correspondence between (a) a Hamilton cycle in $\Xi(4)$, and (b) a spanning tree in $\Xi_4(4)$. The resulting SP-cycle is the aperiodic SP-cycle 432142132413423412312431 from Fig. 1(a)

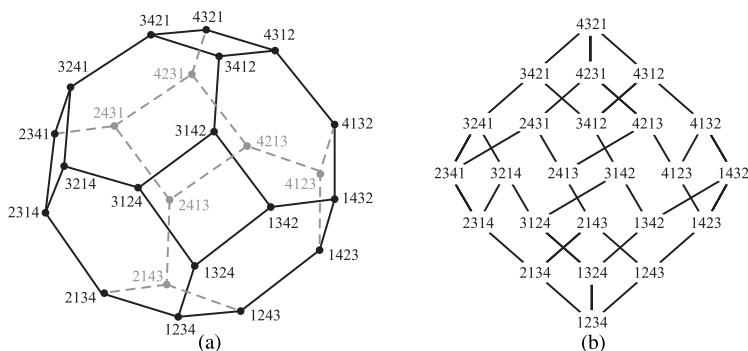


Fig. 7 (a) The permutohedron $\mathbb{P}(4)$, and (b) the weak Bruhat order on S_n

Similarly, the following theorem was previously observed in [13] (Lemma 4.1). The simple connected undirected graph $\Xi_{n-1}(n)$ is obtained from $\Xi(n)$ by contracting each directed cycle using $n-1$ successive σ_{n-1} arcs into a single vertex.

Theorem 5 [13] *Max-weight SP-cycles of $\Pi(n)$ are in one-to-one correspondence with the spanning trees of $\Xi_{n-1}(n)$.*

2.4 The Permutohedron $\mathbb{P}(n-1)$

In this subsection we show that min-weight periodic SP-cycles correspond to the spanning trees of the $(n-1)$ -permutohedron. The (n) -permutohedron $\mathbb{P}(n)$ is a simple connected graph with $n!$ vertices labeled by $\Pi(n)$, and edges between two vertices if they differ by one of the $n-1$ adjacent-transpositions. Figure 7(a) illustrates $\mathbb{P}(4)$.

Recall that the neighborhood of a vertex in $\Xi_n(n)$ is visualized by its n ‘adjacent’-transpositions around its label in $\Pi(n)$ written circularly (see Fig. 5(b)). The $(n-1)$ -

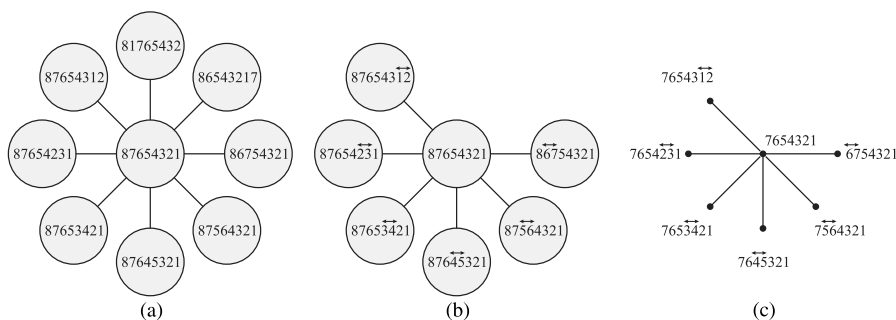


Fig. 8 (a) The neighborhood of the vertex labeled 87654321 in $\Xi_n(n)$, (b) removing the two edges for the ‘adjacent’-transpositions involving 8, and (c) removing 8 from each vertex label to obtain the neighborhood of the vertex labeled 7654321 in $\mathbb{P}(7)$. The adjacent-transpositions in (b) and (c) are highlighted

permutohedron is obtained from $\Xi_n(n)$ by removing the edges for the two ‘adjacent’-transpositions involving n , and then by removing the leading n from each vertex labels. Figure 8 illustrates this transformation for the vertex labeled 87654321 in $\Xi_8(8)$ and the corresponding vertex labeled 7654321 in the (7)-permutohedron. Figure 8(a) is identical to Fig. 5(b) except the labels are written linearly. The following lemma relates periodic SP-cycles to the σ_{n-1} arcs in its Hamilton cycle.

Lemma 1 *An SP-cycle of $\Pi(n)$ is periodic if and only if its Hamilton cycle in $\Xi(n)$ does not follow a σ_{n-1} arc from a node whose label begins or ends with n .*

Proof Let U be an SP-cycle of $\Pi(n)$ and H be its Hamiltonian cycle in $\Xi(n)$. If each node labeled $\mathbf{a} \in \Pi(n)$ with $a_1 = n$ or $a_n = n$ is followed by a σ_n arc in H , then n is the label of every n th arc in H , and so U is periodic. The other direction has two cases. If a node labeled $\mathbf{a} \in \Pi(n)$ with $a_1 = n$ is followed by a σ_{n-1} arc in H , then n will appear as the first and last label among n consecutive arcs in H . If a node labeled $\mathbf{a} \in \Pi(n)$ with $a_n = n$ is followed by a σ_{n-1} arc in H , then n will not appear as an arc label amongst n consecutive arcs in H . In both cases, U is not periodic. $\square$

Now we prove that min-weight periodic SP-cycles correspond to spanning trees of permutohedron. To illustrate this fact, notice that the spanning tree of $\Xi_4(4)$ in Fig. 6(b) is not a spanning tree of $\mathbb{P}(3)$, and so its min-weight SP-cycle is aperiodic.

Theorem 6 *Min-weight periodic SP-cycles of $\Pi(n)$ are in one-to-one correspondence with the spanning trees of $\mathbb{P}(n-1)$.*

Proof Theorem 4 proved that min-weight SP-cycles of $\Pi(n)$ are in one-to-one correspondence with the spanning trees of $\Xi_n(n)$. Therefore, by Lemma 1 we must prove that the two types of ‘missing’ edges in $\mathbb{P}(n-1)$ correspond to the σ_{n-1} arcs in $\Xi(n)$ that originate from nodes whose label begins or ends with n .

Let $\mathbf{a} \in \Pi(n-1)$ and define $\mathbf{b} \in \Pi(n-1)$ and $\mathbf{c} \in \Pi(n)$ as follows

$$\mathbf{b} := a_{n-1} a_1 \cdots a_{n-2}$$

$$\mathbf{c} := a_1 \cdots a_{n-2} n a_{n-1}$$

Notice $(n\mathbf{a}, n\mathbf{b})$ is an edge in $\Xi_n(n)$. Since $\mathbf{c}$ is a rotation of $n\mathbf{b}$, this edge corresponds to the σ_{n-1} arc $(n\mathbf{a}, \mathbf{c})$ in $\Xi(n)$. However, the edge $(\mathbf{a}, \mathbf{b})$ is not in $\mathbb{P}(n-1)$.

Let $\mathbf{a} \in \Pi(n-1)$ and define $\mathbf{b} \in \Pi(n-1)$ and $\mathbf{c} \in \Pi(n)$ as follows

$$\mathbf{b} := n a_2 \cdots a_{n-1} a_1$$

$$\mathbf{c} := a_2 \cdots a_{n-1} a_1 n.$$

Notice $(n\mathbf{a}, n\mathbf{b})$ is an edge in $\Xi_n(n)$. Since $\mathbf{c}$ is a rotation of $n\mathbf{b}$, this edge corresponds to the σ_{n-1} arc $(n\mathbf{a}, \mathbf{c})$ in $\Xi(n)$. However, the edge $(\mathbf{a}, \mathbf{b})$ is not in $\mathbb{P}(n-1)$.

Therefore, spanning trees of $\mathbb{P}(n-1)$ correspond to Hamilton cycles in $\Xi(n)$ that do not use σ_{n-1} arcs from nodes whose label begins or ends with n . $\square$

An *inversion* in $\mathbf{a} \in \Pi(n)$ is a pair (i, j) with $i < j$ and $a_i > a_j$. The *weak Bruhat order* of $\mathbb{S}_n$ is the transitive closure of the cover relation $\prec$, where $\mathbf{a} \prec \mathbf{b}$ if $\mathbf{a}$ and $\mathbf{b}$ differ by an adjacent-transposition and $\mathbf{a}$ has one fewer inversion than $\mathbf{b}$. The order is visualized by drawing an edge from $\mathbf{b}$ down to $\mathbf{a}$ if $\mathbf{a} \prec \mathbf{b}$. The order for $n = 4$ is given in Figure (b) and is respected by the embedding of $\mathbb{P}(4)$ in Fig. 7(a). In Sect. 3, the spanning trees of the permutohedron can also be viewed as tree sub-posets of the weak Bruhat order of $\mathbb{S}_n$.

3 Explicit Constructions

In Sect. 3.2 we define the min-weight periodic SP-cycles $B(n)$ and $C(n)$ by providing two spanning trees of the permutohedron in Sect. 3.1.

3.1 Spanning Trees: Decrementing $\mathcal{H}(n)$ and Decreasing $\mathcal{S}(n)$

In this subsection we define two spanning trees of the permutohedron. The spanning trees are ‘rooted’ at the vertex labeled $n n-1 \cdots 1$, and the parent-child relations in the trees depend on the following definitions for a given $\mathbf{a} \in \Pi(n)$:

- The *decrementing prefix* is the longest prefix of the form $n n-1 \cdots j$ and the *incrementing symbol* is $j-1$.
- The *decreasing prefix* is the longest prefix $a_1 a_2 \cdots a_k$ such that $a_1 > a_2 > \cdots > a_k$ and the *increasing symbol* is a_{k+1} .

The following pair of examples illustrate: (i) values in the (decrementing/decreasing) prefix differ by (one/at least one), (ii) the (decrementing/decreasing) prefix begins with $(n/\text{any symbol})$, and (iii) the (incrementing/increasing) symbol has the next (value/index) compared to the (decrementing/decreasing) prefix

incrementing symbol	increasing symbol	incrementing symbol	increasing symbol
876241 $\overline{5}$ 3	8762 $\overline{4}$ 153	654321 $\overline{8}$ 7	654321 $\overline{8}$ 7.
decrementing prefix	decreasing prefix		decreasing prefix

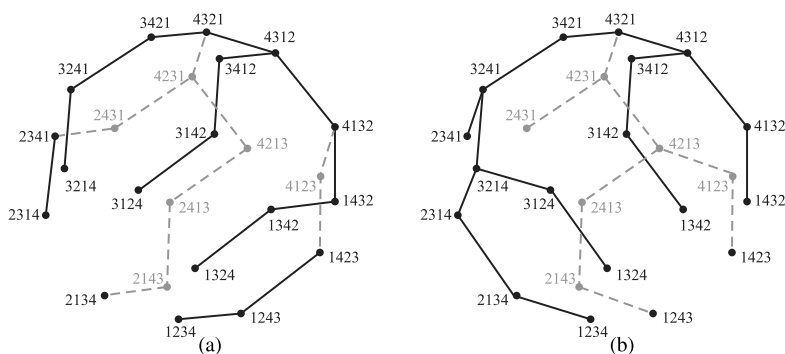


Fig. 9 (a) The decrementing spanning tree $\mathcal{H}(4)$, and (b) the decreasing spanning tree $\mathcal{S}(4)$ of $\mathbb{P}(4)$

In the case of $\mathbf{a} = n\ n-1\ \dots\ 1$, the decrementing and decreasing prefixes include the entire string, and the incrementing symbol and increasing symbol are undefined.

In the *decrementing spanning tree* $\mathcal{H}(n)$, the parent of a non-root vertex is obtained by swapping the incrementing symbol in its label to the left. In the *decreasing spanning tree* $\mathcal{S}(n)$, the parent of a non-root vertex is obtained by swapping the increasing symbol in its label to the left. Since each non-root vertex has a unique parent whose label has one fewer inversion, $\mathcal{H}(n)$ and $\mathcal{S}(n)$ are spanning trees of $\mathbb{P}(n)$ that respect the weak Bruhat order on $\mathbb{S}_n$. Figure 9 shows (a) $\mathcal{H}(4)$ and (b) $\mathcal{S}(4)$.

Now we characterize the children of vertices in these spanning trees. In both trees we distinguish two types of children. Figure 10 illustrates the neighborhood of the vertex in (a) $\mathcal{H}(8)$ and (b) $\mathcal{S}(8)$ with previously discussed label 87624153.

In $\mathcal{H}(n)$ the children of vertex $\mathbf{a} \in \Pi(n)$ are obtained as follows:

- If a_i is in the decrementing prefix and is neither the first nor last symbol in $\mathbf{a}$, then a *decrementing child* is obtained by swapping a_i to the right,
- If a_i is the incrementing symbol and $i < n$, then the *incrementing child* is obtained by swapping a_i to the right.

In $\mathcal{S}(n)$ the children of vertex $\mathbf{a} \in \Pi(n)$ are obtained as follows:

- If a_i is in the decreasing prefix and is neither the first nor last symbol in this prefix, then a *decreasing child* is obtained by swapping a_i to the right,
- If a_i is the increasing symbol and $i < n$ and $a_{i-1} > a_{i+1}$, then the *increasing child* is obtained by swapping a_i to the right.

The symbol for $\mathcal{H}(n)$ was chosen for its relationship to ‘half-hunts’. This bell-ringing term refers to a sweeping motion made by a symbol through a permutation. Figure 9(b) illustrates the left-to-right motion of the symbol 4 in the paths rooted from vertices whose labels begin with 4 in $\mathcal{H}(4)$. The symbol for $\mathcal{S}(n)$ was chosen for its relationship to ‘scuts’. The *scut* of $\mathbf{a} \in \Pi(n)$ is the shortest suffix of $\mathbf{a}$ that is not also a suffix of $n\ n-1\ \dots\ 1$ (see Williams [17]). Figure 9(c) illustrates that subtrees of $\mathcal{S}(4)$ contain each of the scuts: 421, 41, 4, 31, 3, and 2.

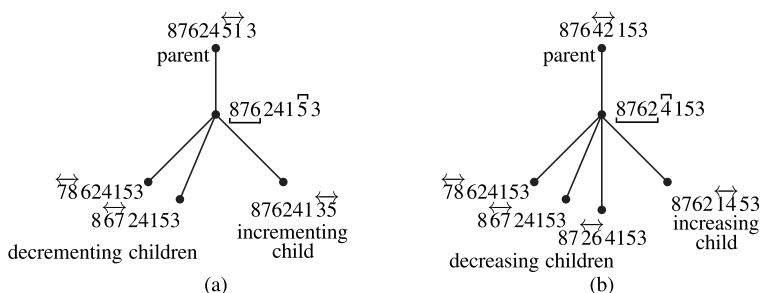


Fig. 10 The neighborhood of the vertex 87624153 in (a) the decrementing spanning tree $\mathcal{H}(8)$, and (b) the decreasing spanning tree $\mathcal{S}(8)$. The decrementing prefix and incrementing symbol of 87624153 are shown in (a), and the decreasing prefix and increasing symbol of 87624153 are shown in (b)

3.2 SP-cycles: Bell-Ringer $B(n)$ and Cool-lex $C(n)$

From Theorem 6, the spanning trees defined in this section correspond to min-weight periodic SP-cycles. The SP-cycle corresponding the decrementing spanning tree $\mathcal{H}(n-1)$ is the *bell-ringer SP-cycle* $B(n)$ of $\Pi(n)$. The SP-cycle corresponding the decreasing spanning tree $\mathcal{S}(n-1)$ is the *cool SP-cycle* $C(n)$ of $\Pi(n)$. The SP-cycles are named for their relationship to ‘half-hunts’ and their sub-permutations, respectively. The rest of this article focuses on decoding the SP-cycles, generating the SP-cycles and their sub-permutations, the binary representation of $B(n)$, and ranking $B(n)$.

4 Decoding the SP-cycles

This section describes how to efficiently decode the SP-cycles defined in Sect. 3. We provide decoding rules that take an arbitrary permutation in $\Pi(n)$ and determines the change, σ_n or σ_{n-1} , that creates the SP-cycle’s next permutation. The rules are *memoryless* since they can be applied in $O(n)$ -time without requiring any additional storage. The rules are presented in Theorems 7 and 8, and are illustrated by Table 1.

If $\mathbf{a} \in \Pi(n)$ and $a_h = n$, the *decrementing substring* is the decrementing prefix of $a_h a_{h+1} \cdots a_n$ and the *decreasing substring* is the decreasing prefix of $a_h a_{h+1} \cdots a_n$. For example, the decrementing and decreasing substrings in $32518764 \in \Pi(8)$ are 876 and 8764, respectively.

Theorem 7 Suppose $\mathbf{a} \in \Pi(n)$ where $m = \max(a_1, a_n)$ and d is the minimum value in its decrementing substring. The permutation of $B(n)$ that follows $\mathbf{a}$ is

- $\mathbf{a}\sigma_{n-1}$ if $d-1 \leq m < n$,
- $\mathbf{a}\sigma_n$ otherwise.

Proof Let $n\mathbf{b}$ be the rotation of $\mathbf{a}$ beginning with n , and let $n\mathbf{c}$ be the rotation of $\mathbf{a}\sigma_{n-1}$ beginning with n . By these choices, $e = (n\mathbf{b}, n\mathbf{c})$ is an edge in $\Xi_n(n)$. Now observe that $\mathbf{a}$ is followed by $\mathbf{a}\sigma_{n-1}$ in $B(n)$ if and only if e is in $\mathcal{H}(n-1)$. Therefore, we

Table 1 Memoryless decoding rules for the bell-ringer SP-cycle in (a)–(f), and the cool SP-cycle in (g)–(l) with $n = 4$. The SP-cycles are given in (a) and (g), their permutations in (b) and (h), the difference between successive permutations in (c) and (i), the values used in the decoding rules in (d)–(e) and (j)–(k), and the reason for applying σ_n according to the two theorems in (f) and (l)

B(4)	$\Pi(4)$	next	m	d	Theorem 7	C(4)	$\Pi(4)$	next	m	d	Theorem 8
4	4321	σ_4	4	1	$m = n$	4	4321	σ_4	4	4	$m = n$
3	3214	σ_4	4	4	$m = n$	3	3214	σ_4	4	4	$m = n$
2	2143	σ_3	3	3		2	2143	σ_3	3	4	
1	1423	σ_3	3	4		1	1423	σ_3	3	3	
4	4213	σ_4	4	4	$m = n$	4	4213	σ_4	4	3	$m = n$
2	2134	σ_4	4	4	$m = n$	2	2134	σ_4	4	4	$m = n$
1	1342	σ_4	2	4	$d - 1 > m$	1	1342	σ_3	2	4	
3	3421	σ_3	3	4		3	3412	σ_4	3	3	$d = n - 1$ and $a_1 > a_{n-1}$
4	4231	σ_4	4	4	$m = n$	4	4123	σ_4	4	2	$m = n$
2	2314	σ_4	4	4	$m = n$	1	1234	σ_4	4	4	$m = n$
3	3142	σ_3	3	4		2	2341	σ_3	2	4	
1	1432	σ_3	2	2		3	3421	σ_3	3	4	
4	4312	σ_4	4	3	$m = n$	4	4231	σ_4	4	2	$m = n$
3	3124	σ_4	4	4	$m = n$	2	2314	σ_4	4	4	$m = n$
1	1243	σ_3	3	3		3	3142	σ_3	3	4	
2	2413	σ_3	3	4		1	1432	σ_3	2	4	
4	4123	σ_4	4	4	$m = n$	4	4312	σ_4	4	3	$m = n$
1	1234	σ_4	4	4	$m = n$	3	3124	σ_4	4	4	$m = n$
2	2341	σ_4	2	4	$d - 1 > m$	1	1243	σ_3	3	4	
3	3412	σ_3	3	4		2	2413	σ_4	3	3	$d = n - 1$ and $a_1 > a_{n-1}$
4	4132	σ_4	4	4	$m = n$	4	4132	σ_4	4	2	$m = n$
1	1324	σ_4	4	4	$m = n$	1	1324	σ_4	4	4	$m = n$
3	3241	σ_3	3	4		3	3241	σ_3	3	4	
2	2431	σ_3	2	3		2	2431	σ_3	2	4	
(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)

must prove that e is $\mathcal{H}(n - 1)$ if and only if $d - 1 \leq m < n$. This is proven by the following three observations:

1. $\mathbf{c}$ is a decrementing child of $\mathbf{b}$ if and only if $d \leq m < n$.
2. $\mathbf{c}$ is the incrementing child of $\mathbf{b}$ if and only if $d - 1 = m < n$ and $a_n = d - 1$.
3. $\mathbf{c}$ is the parent of $\mathbf{b}$ if and only if $d - 1 = m < n$ and $a_1 = d - 1$. □

Theorem 8 Suppose $\mathbf{a} \in \Pi(n)$ where $m = \max(a_1, a_n)$ and d is the last index in its decreasing substring. The permutation of $\mathbf{C}(n)$ that follows $\mathbf{a}$ is

- $\mathbf{a}\sigma_{n-1}$ if $m < n$ and either (i) $d = n$ or (ii) $d = n - 1$ and $a_1 < a_{n-1}$,
- $\mathbf{a}\sigma_n$ otherwise.

Proof Let $n\mathbf{b}$ be the rotation of $\mathbf{a}$ beginning with n , and let $n\mathbf{c}$ be the rotation of $\mathbf{a}\sigma_{n-1}$ beginning with n . By these choices, $e = (n\mathbf{b}, n\mathbf{c})$ is an edge in $\Xi_n(n)$. Now observe that $\mathbf{a}$ is followed by $\mathbf{a}\sigma_{n-1}$ in $\mathbf{C}(n)$ if and only if e is in $\mathcal{S}(n-1)$. Therefore, we must prove that $(\mathbf{b}, \mathbf{c})$ is an edge in $\mathcal{S}(n-1)$ if and only if $m < n$ and either (i) $d = n$ or (ii) $d = n-1$ and $a_1 < a_{n-1}$. This is proven by the following three observations:

1. $\mathbf{c}$ is a decreasing child of $\mathbf{b}$ if and only if $m < n$ and $d = n$ and $a_1 < a_n$.
2. $\mathbf{c}$ is the increasing child of $\mathbf{b}$ if and only if $m < n$ and $d = n-1$ and $a_1 < a_{n-1}$.
3. $\mathbf{c}$ is the parent of $\mathbf{b}$ if and only if $m < n$ and $d = n$ and $a_1 > a_n$. $\square$

Theorems 7 and 8 provide *memoryless algorithms* that require $O(n)$ -time and $O(1)$ -memory to generate successive (i) symbols in $\mathbf{B}(n)$ or $\mathbf{C}(n)$, (ii) substrings or permutations in $\mathbf{B}(n)$ or $\mathbf{C}(n)$, or (iii) bits in the binary representation of $\mathbf{B}(n)$ or $\mathbf{C}(n)$.

5 Generating the SP-cycles

In Sects. 5.2 and 5.3 we determine the sub-permutations of $\mathbf{B}(n)$ and $\mathbf{C}(n)$, respectively. This leads to efficient algorithms in Sect. 5.4 that generate successive blocks of the SP-cycles in constant amortized time. In Sect. 5.5 we reverse our investigation by discussing when an order of $\Pi(n-1)$ can be ‘recycled’ into an SP-cycle for $\Pi(n)$. Several useful identities are first presented in Sect. 5.1.

5.1 Identities using σ and τ

This subsection gives identities that facilitate our investigation of sub-permutations. Each identity applies a series of n permutations in $\{\sigma_n, \sigma_{n-1}\}$ to a string of length n , and equates this to applying σ and/or τ to a string of length $n-1$. The first identity is a special case of the second identity, but is stated separately due to its frequent use.

Lemma 2 *The following identities hold for any $\mathbf{a} = a_1 \cdots a_{n-1}$:*

1. $(n\mathbf{a})\sigma_n^2\sigma_{n-1}^{n-2} = n(\mathbf{a}\sigma_{n-1})$,
2. $(n\mathbf{a})\sigma_n^2\sigma_{n-1}^i\sigma_n^{n-i-2} = n(\mathbf{a}\sigma_{i+1})$ where $1 \leq i \leq n-2$,
3. $(n\mathbf{a})\sigma_n^{j+2}\sigma_{n-1}\sigma_n^{n-j-3} = n(\mathbf{a}\tau_{j+1})$ where $0 \leq j \leq n-3$, and
4. $(n\mathbf{a})\sigma_n^2\sigma_{n-1}^i\sigma_n^{j-i}\sigma_{n-1}\sigma_n^{n-j-3} = n(\mathbf{a}\sigma_{i+1}\tau_{j+1})$ where $i \leq i < j \leq n-2$.

Proof Identities 2 and 3 are proven below on the left and right respectively

$$\begin{array}{ll}
 (n\mathbf{a})\sigma_n^2\sigma_{n-1}^i\sigma_n^{n-i-2} & (n\mathbf{a})\sigma_n^{j+2}\sigma_{n-1}\sigma_n^{n-j-3} \\
 = (a_2 \cdots a_{n-1}na_1)\sigma_{n-1}^i\sigma_n^{n-i-2} & = (a_{j+2} \cdots a_{n-1}na_1 \cdots a_{j+1})\sigma_{n-1}\sigma_n^{n-j-3} \\
 = (a_{i+2} \cdots a_{n-1}na_2 \cdots a_{i+1}a_1)\sigma_n^{n-i-2} & = (a_{j+3} \cdots a_{n-1}na_1 \cdots a_ja_{j+2}a_{j+1})\sigma_n^{n-j-3} \\
 = na_2 \cdots a_{i+1}a_1a_{i+2} \cdots a_{n-1} & = na_1 \cdots a_ja_{j+2}a_{j+1}a_{j+3} \cdots a_{n-1} \\
 = n(\mathbf{a}\sigma_{i+1}) & = n(\mathbf{a}\tau_{j+1}).
 \end{array}$$

Identity 1 is a special case of identity 2. Identity 4 is proven below.

$$\begin{aligned}
 (na)\sigma_n^2\sigma_{n-1}^i\sigma_n^{j-i}\sigma_{n-1}\sigma_n^{n-j-3} \\
 &= (a_2 \cdots a_{n-1}na_1)\sigma_{n-1}^i\sigma_n^{j-i}\sigma_{n-1}\sigma_n^{n-j-3} \\
 &= (a_{i+2} \cdots a_{n-1}na_2 \cdots a_{i+1}a_1)\sigma_n^{j-i}\sigma_{n-1}\sigma_n^{n-j-3} \\
 &= (a_{j+2} \cdots a_{n-1}na_2 \cdots a_{i+1}a_1a_{i+2} \cdots a_{j+1})\sigma_{n-1}\sigma_n^{n-j-3} \\
 &= (a_{j+3} \cdots a_{n-1}na_2 \cdots a_{i+1}a_1a_{i+2} \cdots a_ja_{j+2}a_{j+1})\sigma_n^{n-j-3} \\
 &= na_2 \cdots a_{i+1}a_1a_{i+2} \cdots a_ja_{j+2}a_{j+1}a_{j+3} \cdots a_{n-1} \\
 &= n(a\sigma_{i+1}\tau_{j+1}). \quad \square
 \end{aligned}$$

5.2 7-Order

In this subsection we prove that the sub-permutations of the bell-ringer SP-cycle $B(n)$ follow a new order that we call “seven order” and denote 7-order. It is a recursive method in which every $\mathbf{a} \in \Pi(n-1)$ is expanded into n permutations in $\Pi(n)$ by inserting n into every possible position in $\mathbf{a}$. These insertions are done in a peculiar order that is reminiscent the way the number seven is normally written.

Definition 1 By γ_n we denote the 7-order of $\Pi(n)$. The list γ_1 is the single string 1. The list γ_n is obtained from γ_{n-1} by replacing each $\mathbf{a} \in \Pi(n-1)$ in the list as follows: The first permutation is $n\mathbf{a}$, the second is $\mathbf{a}n$, and the remaining permutations are obtained by moving the n one position to the left until it is in the second position.

By Definition 1, $\gamma_2 = 21, 12$ and $\gamma_3 = 321, 213, 231, 312, 123, 132$ and the first four permutations of γ_4 are 4321, 3214, 3241, 3421. Lemma 3 describes a generation rule for transforming any $\mathbf{x} \in \Pi(n-1)$ into the next permutation in γ_{n-1} . We use $\mathbf{x} \in \Pi(n-1)$ (instead of $\mathbf{a} \in \Pi(n)$) since we refer to both Lemma 3 and Theorem 7 when proving Theorem 9. The rule is illustrated in Table 2(b)–(h) for γ_4 .

Lemma 3 Suppose $\mathbf{x} = x_1 \cdots x_{n-1} \in \Pi(n-1)$ and h is the index such that $x_h = n-1$. If $h = 2$, then let $x_i := p$ be the minimum value in the decrementing prefix of $x_2 \cdots x_{n-1}$ and r be the index such that $x_r := p-1$. The permutation that follows $\mathbf{x}$ in γ_{n-1} is

$$\mathbf{y} = \begin{cases} \mathbf{x}\tau_{h-1} & \text{if } h > 2 \\ \mathbf{x}\sigma_{n-1} & \text{if } h = 1 \text{ or } (h = 2 \text{ and } r = 1) \\ \mathbf{x}\sigma_i\tau_{r-1} & \text{otherwise } (h = 2 \text{ and } r > 1). \end{cases}$$

(4a)

(4b)

(4c)

Proof We verify that (4) correctly follows Definition 1. When $n-1$ is not in the first two positions of $\mathbf{x}$, it is moved one position to the left by (4a). When $n-1$ is in the first position of $\mathbf{x}$, it is moved to the last position (4b). When $n-1$ is in the second position of $\mathbf{x}$, there are two cases. In both cases the decrementing prefix of $x_2 \cdots x_{n-1}$

Table 2 Generating the bell-ringer SP-cycle in (a)–(h) and the cool SP-cycle in (i)–(m) with $n = 5$. The SP-cycles are given in (a) and (i), their sub-permutations in (b) and (j), the difference between successive sub-permutations in (c) and (k), the values used in the generation rules in (d)–(g) and (l), and the reason for applying each generation rule in (h) and (m)

B(5)	γ_4	next	h	p	i	r	Lemma 3	C(5)	$\mathcal{C}_4$	next	p	Definition 2
54321	4321	σ_4	1				$h = 1$	54321	4321	σ_4	4	$p = n - 1$
53214	3214	τ_3	4				$h > 2$	53214	3214	σ_3	3	$x_1 > x_p$
53241	3241	τ_2	3				$h > 2$	52134	2134	σ_2	2	$x_1 > x_p$
53421	3421	σ_4	2	4	2	1	$r = 1$	51234	1234	σ_3	2	$x_1 < x_p$
54213	4213	σ_4	1				$h = 1$	52314	2314	σ_3	3	$x_1 > x_p$
52134	2134	τ_3	4				$h > 2$	53124	3124	σ_2	2	$x_1 > x_p$
52143	2143	τ_2	3				$h > 2$	51324	1324	σ_4	3	$x_1 < x_p$
52413	2413	$\sigma_2 \tau_3$	2	4	2	4	$r > 1$	53241	3241	σ_2	2	$x_1 > x_p$
54231	4231	σ_4	1				$h = 1$	52341	2341	σ_3	2	$x_1 < x_p$
52314	2314	τ_3	4				$h > 2$	53421	3421	σ_4	4	$p = n - 1$
52341	2341	τ_2	3				$h > 2$	54213	4213	σ_3	3	$x_1 > x_p$
52431	2431	σ_4	2	3	3	1	$r = 1$	52143	2143	σ_2	2	$x_1 > x_p$
54312	4312	σ_4	1				$h = 1$	51243	1243	σ_3	2	$x_1 < x_p$
53124	3124	τ_3	4				$h > 2$	52413	2413	σ_3	3	$x_1 > x_p$
53142	3142	τ_2	3				$h > 2$	54123	4123	σ_2	2	$x_1 > x_p$
53412	3412	σ_3	2	4	2	1	$r = 1$	51423	1423	σ_4	3	$x_1 < x_p$
54123	4123	σ_4	1				$h = 1$	54231	4231	σ_2	2	$x_1 > x_p$
51234	1234	τ_3	4				$h > 2$	52431	2431	σ_4	4	$p = n - 1$
51243	1243	τ_2	3				$h > 2$	54312	4312	σ_3	3	$x_1 > x_p$
51423	1423	$\sigma_2 \tau_3$	2	4	2	4	$r > 1$	53142	3142	σ_2	2	$x_1 > x_p$
54132	4132	σ_4	1				$h = 1$	51342	1342	σ_3	2	$x_1 < x_p$
51324	1324	τ_3	4				$h > 2$	53412	3412	σ_3	3	$x_1 > x_p$
51342	1342	τ_2	3				$h > 2$	54132	4132	σ_2	2	$x_1 > x_p$
51432	1432	σ_4	2	2	4	1	$r = 1$	51432	1432	σ_4	4	$p = n - 1$
(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)	

is moved one position to the left by (4b) or (4c). Then the next largest symbol $p - 1$ is either moved one position to the left by (4c) (if $p - 1$ was not in the first position of $\mathbf{x}$) or into the last position by (4b) (if $p - 1$ was in the first position of $\mathbf{x}$). $\square$

Theorem 9 Sub-permutations of the bell-ringer SP-cycle $B(n)$ are in 7-order 7_{n-1} .

Proof Suppose $\mathbf{x}$ is followed by $\mathbf{y}$ in 7_{n-1} and h, i, p , and r are defined according to Lemma 3. We must show that $n\mathbf{x}n\mathbf{y}$ appears in $B(n)$. To do this we prove that n applications of $B(n)$'s decoding rule in Theorem 7 will transform $n\mathbf{x}$ into $n\mathbf{y}$. When considering these applications we let $\mathbf{a}$ be the 'current' permutation (starting from $\mathbf{a} = n\mathbf{x}$) and define m and d according to Theorem 7.

Case one: $h > 2$. The first two applications are σ_n since $m = n$. The next $h - 2$ applications are σ_n since $d = n$ and $m < n - 1$. The next application is σ_{n-1} since

$d = n$ and $m = n - 1$. The remaining $n - h$ applications are σ_{n-1} since $d = n$ and $m < n - 1$. Thus, $n\mathbf{x}$ has been transformed into $(n\mathbf{x})\sigma_n^h\sigma_{n-1}\sigma_n^{n-h} = n(\mathbf{x}\tau_{h-1})$ by identity 3 in Lemma 2. This is correct since $\mathbf{y} = \mathbf{x}\tau_{h-1}$ by (4a).

Case two: $h = 1$. The first two applications are σ_n since $m = n$. The next application is σ_{n-1} since $m = n - 1$ and $d = n - 1$. The remaining $n - 3$ applications are σ_{n-1} since $m = n - 1$ and $d = n$. Thus, $n\mathbf{x}$ has been transformed into $(n\mathbf{x})\sigma_n^2\sigma_{n-1}^{n-2} = n(\mathbf{x}\sigma_{n-1})$ by identity 1 in Lemma 2. This is correct since $\mathbf{y} = \mathbf{x}\sigma_{n-1}$ by the $h = 1$ condition in (4b).

Case three: $h = 2$. The first two applications are σ_n since $m = n$. The next $i - 1$ applications are σ_{n-1} since $m = n - 1, n - 2, n - 3, \dots, p$ and $d = n, n - 1, n - 2, \dots, p + 1$ during these applications, respectively. The current string now has the suffix $x_2x_3 \dots x_ix_1$. We also know that $x_2x_3 \dots x_i = n - 1n - 2 \dots p$ and $x_r = p - 1$. The proof of this case now splits into two subcases.

Subcase one: $r = 1$. In this subcase, the decrementing substring of the current string is $x_2x_3 \dots x_ix_1$ and its minimum value is $p - 1$. If $i = n - 1$, then all n applications have already been considered. Otherwise, the next application is σ_{n-1} since $m = p - 1$ and $d = p - 1$. The remaining $n - i - 2$ applications are σ_{n-1} since $m = p - 1$ and $d = p$. Thus, $n\mathbf{x}$ has been transformed into $(n\mathbf{x})\sigma_n^2\sigma_{n-1}^{n-2} = n(\mathbf{x}\sigma_{n-1})$ by identity 1 in Lemma 2. This is correct since $\mathbf{y} = \mathbf{x}\sigma_{n-1}$ by the $h = 2$ and $r = 1$ condition in (4b).

Subcase two: $r > 1$. In this subcase, the decrementing substring of the current string is $x_2x_3 \dots x_i$ and its minimum value is p . Therefore, the next $r - i - 1$ applications are σ_n since $m < p - 1$ and $d = p$. The next application is σ_{n-1} since $m = p - 1$ and $d = p$. The remaining $n - r - 1$ applications are σ_n since $m < p - 1$ and $d = p$. Thus, $n\mathbf{x}$ has been transformed into $(n\mathbf{x})\sigma_n^2\sigma_{n-1}^{i-1}\sigma_n^{r-i-1}\sigma_{n-1}\sigma_n^{n-r-1} = n(\mathbf{x}\sigma_i\tau_{r-1})$ by Identity 4 in Lemma 2. This is correct since $\mathbf{y} = \mathbf{x}\sigma_i\tau_{r-1}$ by (4c). $\square$

5.3 Cool-lex Order

In this subsection we prove that the sub-permutations of the cool SP-cycle $C(n)$ follow a “cool-lex order”. The order $\mathfrak{C}_n$ is a *cyclic prefix-shift Gray code* of $\Pi(n)$, meaning that successive permutations differ by σ_i for some $i \in \langle n \rangle$. By convention, $\mathfrak{C}_n$ begins with $n \dots 1$. Definition 2 gives the correct σ_i for transforming any $\mathbf{x} \in \Pi(n - 1)$ into the next permutation in $\mathfrak{C}_{n-1}$. We use $\mathbf{x} \in \Pi(n - 1)$ (instead of $\mathbf{a} \in \Pi(n)$) since we refer to both Definition 2 and Theorem 8 when proving Theorem 10. The rule is illustrated in Table 2(j)–(m) for $\mathfrak{C}_4$.

Definition 2 (Cool-lex order) Suppose $\mathbf{x} \in \Pi(n - 1)$ and p is the last index of the decreasing prefix of $x_2 \dots x_n$. The permutation that follows $\mathbf{x}$ in $\mathfrak{C}_{n-1}$ is

$$\mathbf{y} := \begin{cases} \mathbf{x}\sigma_{n-1} & \text{if } p = n - 1 \\ \mathbf{x}\sigma_p & \text{if } p < n - 1 \text{ and } x_1 > x_p \\ \mathbf{x}\sigma_{p+1} & \text{otherwise } (p < n - 1 \text{ and } x_1 < x_p). \end{cases} \quad \begin{matrix} (5a) \\ (5b) \\ (5c) \end{matrix}$$

We refer to $\mathfrak{c}_n$ as the *cool-lex order* of $\Pi(n)$. (Definition 2 is actually the inverse of Definition 2.7 in [17], so $\mathfrak{C}_n$ would be described as *reverse cool-lex order* in [17].)

Theorem 10 *Sub-permutations of the cool SP-cycle $C(n)$ are in cool-lex order $\mathfrak{C}_{n-1}$.*

Proof Suppose $\mathbf{x}$ is followed by $\mathbf{y}$ in $\mathfrak{C}_{n-1}$ and p is defined according to Definition 2. We must show that $n\mathbf{x}n\mathbf{y}$ appears in $C(n)$. To do this we prove that n applications of $C(n)$'s decoding rule in Theorem 8 will transform $n\mathbf{x}$ into $n\mathbf{y}$. When considering these applications we let $\mathbf{a}$ be the ‘current’ permutation (starting from $\mathbf{a} = n\mathbf{x}$) and define m and d according to Theorem 8.

The first two applications are σ_n since $m = n$. During the next $p - 1$ applications, notice that the current string has suffix $x_1, x_2x_1, x_2x_3x_1, \dots, x_2x_3 \cdots x_px_1$, respectively. Also, $x_2 > x_3 > \cdots > x_p$. Therefore, either (i) $d = n$ or (ii) $d = n - 1$ and $a_1 < a_{n-1}$. Therefore, these $p - 1$ applications are σ_n . The proof is now divided into cases that correspond to the cases in Definition 2.

Case one: $p = n - 1$. In this case all n applications have already been considered. Thus, $n\mathbf{x}$ has been transformed into $(n\mathbf{x})\sigma_n^2\sigma_{n-1}^{n-2} = n(\mathbf{x}\sigma_{n-1})$ by identity 1 in Lemma 2. This is correct since $\mathbf{y} = \mathbf{x}\sigma_{n-1}$ by (5a).

Case two: $p < n - 1$ and $x_1 < x_p$. In this case $d = n$ since the current string has suffix $x_2x_3 \cdots x_px_1$ and $x_2 > x_3 > \cdots > x_p > x_1$. Therefore, the next application is σ_{n-1} . The remaining $n - p - 2$ applications are σ_n since $d = n - 2, n - 3, \dots, p + 1$ during these applications, respectively. Thus, $n\mathbf{x}$ has been transformed into $(n\mathbf{x})\sigma_n^2\sigma_{n-1}^p\sigma_n^{n-p-2} = n(\mathbf{x}\sigma_{p+1})$ by identity 2 in Lemma 2. This is correct since $\mathbf{y} = \mathbf{x}\sigma_p$ by (5b).

Case three: $p < n - 1$ and $x_1 > x_p$. In this case $d = n - 1$ since the current string has suffix $x_2x_3 \cdots x_px_1$ and $x_2 > x_3 > \cdots > x_p$ and $x_p < x_1$. Also notice that the first symbol in the current string is x_{p+1} . Therefore, $a_1 = x_{p+1} > x_p = a_{n-1}$ by the definition of p . Therefore, the next application is σ_n . The remaining $n - p - 2$ applications are σ_n since $d = n - 2, n - 3, \dots, p + 1$ during these applications, respectively. Thus, $n\mathbf{x}$ has been transformed into $(n\mathbf{x})\sigma_n^2\sigma_{n-1}^{p-1}\sigma_n^{n-p-1} = n(\mathbf{x}\sigma_p)$ by identity 2 in Lemma 2. This is correct since $\mathbf{y} = \mathbf{x}\sigma_p$ by (5c). $\square$

5.4 Algorithms for Generating the SP-cycles

This subsection provides CAT algorithms for generating the SP-cycles and their sub-permutations. Bell7 and Cool are CAT algorithms that generate permutations in 7-order and cool-lex order, respectively. Since these orders are the sub-permutations of the SP-cycles, Bell7 and Cool are also CAT algorithms that generate blocks of the bell-ringer and cool SP-cycle, respectively. This is done by storing the current permutation in an array a whose first entry is never changed. Pseudo-code for Bell7 and Cool appear in Algorithms 1.

Both algorithms generate the sub-permutations recursively. To understand Bell7, recall Definition 1 and notice that line 6 moves symbol m to the end of the array, and line 9 moves it back to its initial location one position at a time. To understand Cool, we must consider the recursive structure of cool-lex order (see Definition 2.4 in [17]). The order starts with the permutation $n - 1 \ n - 2 \ \cdots \ 1$, and then the scuts are ordered by decreasing value of the first symbol, followed by increasing length. For example, consider $\mathfrak{C}_4$ in Table 2(j). The first permutation is 4321, and the scut of every other $\mathbf{a} \in \Pi(4)$ is its shortest suffix that is not also a suffix of 4321. Observe that the scuts—4, 41, 421, 3, 31, 2—are ordered by decreasing value of their first symbol, and then

Algorithm 1 Bell7(2) visits permutations in 7_{n-1} or blocks of $B(n)$. Cool(n) visits permutations in $\mathfrak{C}_{n-1}$ or blocks of $C(n)$. The global array $a_0 a_2 \cdots a_{n-1} := n \ n-1 \cdots 1$ must be initialized in both cases, and n is a global constant in Bell7

Procedure Bell7(m)	Procedure Cool(m)
1: if $m = n$ then	1: visit()
2: visit()	2: for $j \leftarrow 1$ to $m - 2$
3: return	3: shift($j, m - 1$)
4: end	4: for $i \leftarrow m - 2$ down to j
5: Bell7($m + 1$)	5: Cool(i)
6: shift($n - m, n - 1$)	6: $a_i \leftrightarrow a_{i+1}$
7: for $i \leftarrow n - 2$ down to $n - m$	7: end
8: Bell7($m + 1$)	8: end
9: $a_i \leftrightarrow a_{i+1}$	
10: end	

by increasing length. This recursive structure provides the prefix-shift Gray code in Definition 2 by Theorem 2.1 in [17]. The outer loop in Cool creates scuts of length one with first symbol equal to $n-1, n-2, \dots, 2$, and the inner loop increases the length of these scuts until the permutation is returned to its original state of $n-1 \ n-2 \cdots 1$.

To complete the procedures in Algorithms 1, array $a = a_0 a_1 \cdots a_{n-1}$ must be initialized to contain the values $n \ n-1 \cdots 1$, shift(i, j) must replace the array contents $a_i a_{i+1} \cdots a_j$ by $a_{i+1} \cdots a_j a_i$, and the instruction $a_i \leftrightarrow a_{i+1}$ must replace $a_i a_{i+1}$ by $a_{i+1} a_i$. Each visit can output a block of the SP-cycle or a sub-permutation by calling output($a_0 a_1 \cdots a_{n-1}$) or output($a_1 a_2 \cdots a_{n-1}$), respectively.

Now we prove that Bell7 and Cool improve upon the $O((n)!)$ -time generation of the direct SP-cycle D(n) in [13].

Theorem 11 Bell7(2) visits all $(n-1)!$ permutations in 7_{n-1} and all $(n-1)!$ blocks of $B(n)$ in $O((n-1)!)$ -time. Similarly, Cool(n) visits all $(n-1)!$ permutations in $\mathfrak{C}_{n-1}$ and all $(n-1)!$ blocks of $C(n)$ in $O((n-1)!)$ -time.

Proof The correctness of these algorithms was previously discussed.

If $n = 2$, then Bell7(2) makes one visit at an expense of $O(1)$ basic operations. If $n > 2$, then Bell7($n - 1$) makes $n - 1$ calls to visit (from Bell7(n) on lines 5 and 8) at an expense of $O(n - 1)$ basic operations (from lines 6 and 9). Thus, by induction Bell7(2) runs in $O(2! + 3! + \cdots + (n - 1)!)$ -time, or simply $O((n - 1)!)$ -time.

Cool(m) calls visit, and each outer loop iteration makes $m - 1 - j$ recursive calls at cost of $O(m - 1 - j)$ basic operations. Thus, Cool(n) runs in $O((n - 1)!)$ -time. $\square$

5.5 Recycling

Periodic SP-cycles are created by inserting n between successive permutations in an ordering of $\Pi(n-1)$. In this subsection we prove that most previously studied orders of $\Pi(n-1)$ cannot be ‘recycled’ in this way. On the other hand, we prove that the permutations of any SP-cycle can be recycled.

Definition 3 If $P = \mathbf{a}, \mathbf{b}, \dots, \mathbf{z}$ is an ordering of $\Pi(n-1)$, then P is *recyclable* if the circular string $\otimes(P) := \mathbf{nanb} \cdots \mathbf{nz}$ is an SP-cycle.

Theorems 9 and 10 proved that 7-order and cool-lex order are recyclable, respectively. On the other hand, Fig. 1(e) illustrates that 321, 132, 312, 123, 213, 231 is not recyclable due to the invalid substring 141 and repeated copies of 342. Theorem 12 proves that these two types of ‘errors’ are the only obstacles to being recyclable.

Theorem 12 Suppose P is an ordering of $\Pi(n-1)$. P is recyclable if and only if the following two conditions are satisfied:

- If $\mathbf{a}$ is followed by $\mathbf{b}$ in P and $a_i = b_j$, then $i - j \leq 1$.
- If $\mathbf{a}$ is followed by $\mathbf{b}$, and $\mathbf{c}$ is followed by $\mathbf{d}$ in P , and there is a j such that

$$a_{j+1} \cdots a_n b_1 \cdots b_{j-1} = c_{j+1} \cdots c_n d_1 \cdots d_{j-1},$$

then $\mathbf{a} = \mathbf{c}$ (and hence $\mathbf{b} = \mathbf{d}$).

Proof The first condition holds if and only if each substring of $\otimes(P)$ is an $(n-1)$ -permutation of $\langle n \rangle$. The second condition holds if and only if each substring of $\otimes(P)$ is distinct. Therefore, $\otimes(P)$ is an SP-cycle if and only if both conditions hold. $\square$

To illustrate Theorem 12, consider the lexicographic order of $\Pi(3)$ given below

$$123, 132, 213, 231, 312, 321.$$

The first condition in Theorem 12 is not satisfied for $\mathbf{a} = 132$ and $\mathbf{b} = 213$. This is because the symbol $a_3 = b_1 = 2$ moves more than one position to the left from $\mathbf{a}$ to $\mathbf{b}$. Thus, recycling produces the invalid substring 242. Invalid substrings are avoided by the Johnson-Trotter order in (1). However, the second condition of Theorem 12 is not satisfied for $\mathbf{a} = 132$, $\mathbf{b} = 312$, $\mathbf{c} = 312$, $\mathbf{d} = 321$, and $j = 2$. This is because the substring 23 appears in the same position in $\mathbf{ab}$ and $\mathbf{cd}$, and so recycling produces repeated substrings 243. The following lemma proves that recycling produces repeated substrings in every adjacent-transposition Gray code.

Lemma 4 If P is a cyclic adjacent-transposition Gray code for $\Pi(n)$ with $n > 2$, then P is not recyclable.

Proof There exist two successive permutations $\mathbf{a}$ and $\mathbf{b}$ in P such that $\mathbf{b} = \mathbf{a}\tau_1$. Let $\mathbf{c}$ be the permutation that follows $\mathbf{b}$ in P . Notice that $\mathbf{c} \neq \mathbf{b}\tau_1$ (otherwise $\mathbf{c} = \mathbf{a}$). Therefore, $a_3 a_4 \cdots a_n b_1 = b_3 b_4 \cdots b_n c_1$ and so $\otimes(P)$ contains repeated substrings. $\square$

Despite the large number of permutation orders that have been studied [9, 15], the authors are aware of only two previously studied orders that could be recyclable. One of these is the (reverse) cool-lex order from Sect. 5.3. The other is a prefix-shift Gray code of $\Pi(n-1)$ due to Corbett [3] that we denote by $\mathfrak{R}_{n-1}$. The symbol $\mathfrak{R}$ refers to Corbett’s consideration of the “rotator graph” $\overrightarrow{\text{Cay}}(\{\sigma_2, \sigma_3, \dots, \sigma_n\}, \mathbb{S}_n)$.

Conjecture 1 Corbett’s prefix-shift Gray code of permutations [3] is recyclable. That is, $R(n) := \otimes(\mathfrak{R}_{n-1})$ is an SP-cycle.

Although recyclable orders of permutations appear to be rare in the literature, the following theorem proves that the permutations of any SP-cycle are recyclable. For example, the permutations from SP-cycle Fig. 1(a) are 4321, 3214, $\dots$, 1432 and so 5432153214 $\dots$ 51432 is an SP-cycle.

Theorem 13 (Re-recycling) *If U is an SP-cycle of $\Pi(n-1)$ and P is its order of permutations, then $\otimes(P)$ is an SP-cycle of $\Pi(n)$*

Proof P is a $(\sigma_n - \sigma_{n-1})$ -Gray code of $\Pi(n-1)$ by Corollary 1. Therefore, P satisfies the first condition of Theorem 12. To verify the second condition, suppose $\mathbf{a}$ is followed by $\mathbf{b}$, and $\mathbf{c}$ is followed by $\mathbf{d}$, and j is chosen such that

$$a_{j+1} \cdots a_n b_1 \cdots b_{j-1} = c_{j+1} \cdots c_n d_1 \cdots d_{j-1}. \quad (6)$$

Since $\mathbf{b} \in \{\mathbf{a}\sigma_n, \mathbf{a}\sigma_{n-1}\}$ and $\mathbf{d} \in \{\mathbf{c}\sigma_n, \mathbf{c}\sigma_{n-1}\}$ we also have

$$b_1 \cdots b_{j-1} = a_2 \cdots a_j \text{ and } d_1 \cdots d_{j-1} = c_2 \cdots c_j. \quad (7)$$

Observe that (6), (7), and $\mathbf{a}, \mathbf{c} \in \Pi(n)$ imply $\mathbf{a} = \mathbf{c}$ (and hence $\mathbf{b} = \mathbf{d}$). $\square$

Section 6.4 proves that the direct SP-cycle is ‘canonical’ in terms of Theorem 13.

6 Binary Representation of SP-cycles

This section focuses on binary representations. Section 6.1 examines periodic SP-cycles, and Sect. 6.2 determines a recursive expression for $\text{binary}(B(n))$. Section 6.3 gives a loopless algorithm to generate (and output) the ‘blocks’ of $\text{binary}(B(n))$. Section 6.4 uses binary representations for a new characterization of $D(n)$.

Binary representations are visualized by aligning an SP-cycle with a copy of itself rotated counter-clockwise $n-1$ symbols. In these visualizations, 1 or 0 is recorded for aligned symbols that are equal or not equal, respectively. Figure 11(a) illustrates this visualization for $\text{binary}(B(4))$. In general, the *binary representation* of an SP-cycle U for $\Pi(n)$ is the circular string $B = \text{binary}(U)$ of length $n!$ such that

$$b_i := \llbracket u_i = u_{i+n-1} \rrbracket,$$

where the *Iversonian* is defined by $\llbracket \text{true} \rrbracket := 1$ and $\llbracket \text{false} \rrbracket := 0$. As mentioned in Sect. 1, $\text{binary}(U)$ records 0 or 1 when U ’s successive permutations differ by σ_n or σ_{n-1} , respectively.

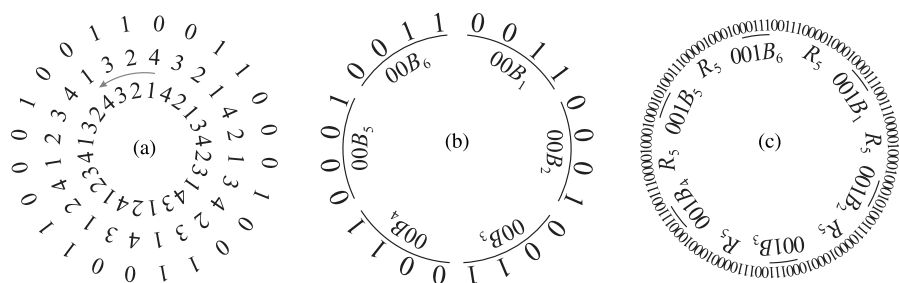


Fig. 11 (a) $\text{binary}(\text{B}(4))$ (outer) is visualized by rotating $\text{B}(4)$ (middle) three positions counter-clockwise (inner), (b) blocks in $\text{binary}(\text{B}(4))$ have the form $00B_i$, and (c) $\text{binary}(\text{B}(5))$ is created by replacing each $00B_i$ in $\text{binary}(\text{B}(4))$ by $R_5 001B_i$

6.1 Binary Representation of Periodic SP-cycles

If U is an SP-cycle for $\Pi(n)$, then the *blocks* of $B = \text{binary}(U)$ are each $b_{i \cdot n+1} \cdots b_{i \cdot (n+1)}$. For example, the blocks of $\text{binary}(\text{B}(4))$ in Fig. 11(a) are 0011, 0001, 0011, 0011, 0001, and 0011, respectively. If U is a periodic SP-cycle, then σ_n is applied in U 's (σ_n, σ_{n-1}) -Gray code whenever n is the first or last symbol in the current permutation. This leads to the following remark that is illustrated in Fig. 11(b) for $\text{binary}(\text{B}(4))$.

Remark 2 If U is a periodic SP-cycle for $\Pi(n)$, then its binary representation can be expressed as

$$\text{binary}(U) = 00B_1 00B_2 \cdots 00B_{(n-1)!},$$

where each B_i has length $n - 2$. In other words, blocks in $\text{binary}(U)$ begin with 00.

The binary representations of the periodic SP-cycles discussed in this article can be further restricted by translating the identities of Lemma 2 into blocks.

Remark 3 If the i th and $(i+1)$ st blocks in a periodic SP-cycle U for $\Pi(n)$ are $n\mathbf{a}$ and $n\mathbf{b}$, respectively, then the i th block in $\text{binary}(U)$ is

1. 001^{n-2} if $\mathbf{b} = \mathbf{a}\sigma_{n-1}$.
2. $001^i 0^{n-i-2}$ if $\mathbf{b} = \mathbf{a}\sigma_{i+1}$ for $1 \leq i \leq n-2$.
3. $000^j 10^{n-j-3}$ if $\mathbf{b} = \mathbf{a}\tau_{j+1}$ for $0 \leq j \leq n-3$.
4. $001^i 0^{j-i} 10^{n-j-3}$ if $\mathbf{b} = \mathbf{a}\sigma_{i+1}\tau_{j+1}$ for $1 \leq i < j \leq n-2$.

The $\Theta(n)$ different blocks in identity 2 imply there are $\Theta(n)$ different blocks in $\text{binary}(\text{C}(n))$. Likewise, the $\Theta(n^2)$ different blocks in identities 1, 3, and 4 imply there are $\Theta(n^2)$ different blocks in $\text{binary}(\text{B}(n))$. Thus, algorithms could pre-compute these blocks when generating $\text{binary}(\text{B}(n))$ or $\text{binary}(\text{C}(n))$. In a broad-word sense each of these blocks can be output in $O(1)$ -time by the following identities

$$\begin{aligned}
(001^{n-2})_2 &= 2^{n-2} - 1 & (001^i 0^{n-i-2})_2 &= 2^{n-2} - 2^{n-2-i} \\
(000^j 10^{n-j-3})_2 &= 2^{n-j-3} & (001^i 0^{j-i} 10^{n-j-3})_2 &= 2^{n-2} - 2^{n-2-i} + 2^{n-j-3}.
\end{aligned}$$

6.2 Binary Representation of the Bell-Ringer SP-cycle

In this subsection we determine a recursive expression for the binary representation of the bell-ringer SP-cycle $B(n)$. Theorem 14 is illustrated for $n = 5$ in Fig. 11(b) and (c). The theorem uses the following string of length $n(n-2)$

$$R_n := \underbrace{001^{n-2}}_{\sigma_{n-1}} \underbrace{0^{n-1}1}_{\tau_{n-2}} \underbrace{0^{n-2}10}_{\tau_{n-3}} \underbrace{0^{n-3}100}_{\tau_{n-4}} \cdots \underbrace{00010^{n-4}}_{\tau_2}. \quad (8)$$

Theorem 14 *The binary representation of the bell-ringer SP-cycle for $\Pi(n)$ is*

$$\text{binary}(B(n)) = R_n 001B_1 R_n 001B_2 \cdots R_n 001B_{(n-2)!}, \quad (9)$$

where $00B_i$ is the i th block in $\text{binary}(B(n-1))$ and R_n is defined in (8).

Proof We prove the following: If $00B$ is the block in $\text{binary}(B(n-1))$ corresponding to consecutive blocks $n-1 \mathbf{a}$ and $n-1 \mathbf{b}$ in $B(n-1)$, then $R_n 001B$ are the blocks in $\text{binary}(B(n))$ corresponding to the blocks from $n n-1 \mathbf{a}$ to $n n-1 \mathbf{b}$ in $B(n)$.

Observe that if 0 and 1 are treated as σ_{n-1} and σ_{n-2} , respectively, then 00 changes $n-1 \mathbf{a}$ into $a_2 \cdots a_{n-2} n-1 a_1$, and B changes $a_2 \cdots a_{n-2} n-1 a_1$ into $n-1 \mathbf{b}$.

Consider the block $n n-1 \mathbf{a}$ in $B(n)$. By Definition 1, $n-1 \mathbf{a}$ is followed in γ_{n-1} by applying $\sigma_{n-1}, \tau_{n-2}, \tau_{n-3}, \dots, \tau_2$. This changes $n-1 \mathbf{a}$ into $a_1 n-1 a_2 \cdots a_{n-2}$ and corresponds to R_n in $\text{binary}(B(n))$ by Remark 3 and (8). Now consider the consecutive blocks $n a_1 n-1 a_2 \cdots a_{n-2}$ and $n n-1 \mathbf{b}$ in $B(n)$. If 0 and 1 are treated as σ_n and σ_{n-1} , respectively, then 001 changes $n a_1 n-1 a_2 \cdots a_{n-2}$ into $a_2 \cdots a_{n-2} n n-1 a_1$ and by our earlier observation B changes $a_2 \cdots a_{n-2} n n-1 a_1$ into $n n-1 \mathbf{b}$. $\square$

6.3 Generating the Binary Representation of the Bell-Ringer SP-cycle

This subsection focuses on generating the blocks of $\text{binary}(B(n))$. $\text{BinaryBellCAT}(n)$ is the standard CAT algorithm for generating the $(n-1)!$ multi-radix numbers in the product space $(n-1) \times \cdots \times 2 \times 1$ in co-lexicographic order (see Algorithm M in Knuth [9] (p. 2)) with additional outputs on lines 8–12. Similarly, $\text{BinaryBell}(n)$ is the standard loopless algorithm for generating the same multi-radix numbers in the reflected Gray code (see Algorithm H in Knuth [9] (p. 20)) except for lines 6–12. Pseudo-code for BinaryBellCAT and BinaryBell appear in Algorithms 2. Table 3 illustrates the progression of values in $\text{BinaryBellCAT}(5)$ and $\text{BinaryBell}(5)$. Notice that the outputs in Table 3(d) and (j) match $\text{binary}(B(5))$ in Fig. 3(c).

Theorem 15 *$\text{BinaryBellCAT}(n)$ and $\text{BinaryBell}(n)$ output the successive blocks in $\text{binary}(B(n))$ in amortized $O(1)$ -time and worst-case $O(1)$ -time, respectively.*

Proof The strings in each output can be computed and output in $O(1)$ -time by Sect. 6.1, so the run-times follow from Algorithm M and Algorithm H in [9].

Algorithm 2 BinaryBellCAT(n) and BinaryBell(n) generate binary($B(n)$)’s blocks

Procedure BinaryBellCAT(n)	Procedure BinaryBell(n)
1: $a_1 \cdots a_{n-1} \leftarrow 0 \cdots 0$	1: $a_1 \cdots a_{n-1} \leftarrow 0 \cdots 0$
2: loop	2: $d_1 \cdots d_{n-1} \leftarrow 1 \cdots 1$
3: $j \leftarrow 1$	3: $f_1 \cdots f_{n-1} \leftarrow 1 \cdots n-1$
4: while $a_j = n-j-1$ and $j < n-1$	4: loop
5: $a_j \leftarrow 0$	5: $j \leftarrow f_1$
6: $j \leftarrow j+1$	6: if $a_j = 0$ or $a_j = n-j-1$ then
7: end	7: $\text{output}(001^{n-2})$
8: if $a_j = 0$ then	8: else if $d_j = 1$ then
9: $\text{output}(001^{n-2})$	9: $\text{output}(001^{j-1}0^{n-a_j-j-1}10^{a_j-1})$
10: else	10: else
11: $\text{output}(001^{j-1}0^{n-a_j-j-1}10^{a_j-1})$	11: $\text{output}(001^{j-1}0^{a_j}10^{n-j-a_j-2})$
12: end	12: end
13: if $j = n-1$ then	13: if $j = n-1$ then
14: return	14: return
15: end	15: end
16: $a_j \leftarrow a_j+1$	16: $f_1 \leftarrow 1$
17: end	17: $a_j \leftarrow a_j + d_j$
	18: if $a_j = 0$ or $a_j = n-j-1$ then
	19: $d_j \leftarrow -d_j$
	20: $f_j \leftarrow f_{j+1}$
	21: $f_{j+1} \leftarrow j+1$
	22: end
	23: end

The first $n-1$ iterations of every n iterations in BinaryBellCAT(n) have $j=1$ and $a_1=0, 1, \dots, n-3$ on line 8, and so these iterations output 001^{n-2} on line 9 followed by $0^{n-1}1, 0^{n-2}10, \dots, 00010^{n-4}$ on line 11. During every n th iteration, j keeps track of the level of recursion, and the 1^{j-1} on line 11 corresponds to the number of times $00B$ has been replaced by $001B$ in (9). Therefore, every n iterations output $R_n 001B$ as required by Theorem 14. BinaryBell(n) is similar to BinaryBellCAT(n) except lines 6–7 account for the two extreme values of a_j , and lines 8–11 account for both increasing and decreasing values in a_j . $\square$

6.4 Binary Representations and Recycling

This subsection investigates the effect of recycling on binary representations and the direct SP-cycle in particular. The following lemma strengthens Theorem 13 by explicitly stating the binary representation of an SP-cycle that is obtained from recycling.

Lemma 5 *If U is an SP-cycle of $\Pi(n-1)$ with permutations P and $\text{binary}(U) = b_1 \cdots b_{(n-1)!}$, then $\text{binary}(\otimes(P))$ is obtained by the morphism $b \rightarrow 001^{n-3}\overline{b}$. That is,*

$$\text{binary}(\otimes(P)) = 001^{n-3}\overline{b_1}001^{n-3}\overline{b_2} \cdots 001^{n-3}\overline{b_{(n-1)!}}. \quad (10)$$

Table 3 Successive values in (a)–(d) BinaryBellCAT(5) and (e)–(j) BinaryBell(5). Mixed-radix numbers are in (a) and (e), blocks of binary(B(5)) in (d) and (j), directions in (f), and focus pointers in (g)

$a_1a_2a_3a_4$	j	a_j	output	$a_1a_2a_3a_4$	$d_1d_2d_3d_4$	$f_1f_2f_3f_4$	j	a_j	output
0000	1	0	00111	0000	+1+1+1+1	1234	1	0	00111
1000	1	1	00001	1000	+1+1+1+1	1234	1	1	00001
2000	1	2	00010	2000	+1+1+1+1	1234	1	2	00010
3000	2	0	00111	3000	-1+1+1+1	2234	2	0	00111
0100	1	0	00111	3100	-1+1+1+1	1234	1	3	00111
1100	1	1	00001	2100	-1+1+1+1	1234	1	2	00001
2100	1	2	00010	1100	-1+1+1+1	1234	1	1	00010
3100	2	1	00101	0100	+1+1+1+1	2234	2	1	00101
0200	1	0	00111	0200	+1-1+1+1	1334	1	0	00111
1200	1	1	00001	1200	+1-1+1+1	1334	1	1	00001
2200	1	2	00010	2200	+1-1+1+1	1334	1	2	00010
3200	3	0	00111	3200	-1-1+1+1	3234	3	0	00111
0010	1	0	00111	3210	-1-1-1+1	1244	1	3	00111
1010	1	1	00001	2210	-1-1-1+1	1244	1	2	00001
2010	1	2	00010	1210	-1-1-1+1	1244	1	1	00010
3010	2	0	00111	0210	+1-1-1+1	2244	2	2	00111
0110	1	0	00111	0110	+1-1-1+1	1244	1	0	00111
1110	1	1	00001	1110	+1-1-1+1	1244	1	1	00001
2110	1	2	00010	2110	+1-1-1+1	1244	1	2	00010
3110	2	1	00101	3110	-1-1-1+1	2244	2	1	00101
0210	1	0	00111	3010	-1+1-1+1	1434	1	3	00111
1210	1	1	00001	2010	-1+1-1+1	1434	1	2	00001
2210	1	2	00010	1010	-1+1-1+1	1434	1	1	00010
3210	4	0	00111	0010	+1+1-1+1	4234	4	0	00111
(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(i)	(j)

Proof Suppose $\mathbf{a}$ is the i th permutation in P and is followed by $\mathbf{c}$. If $b_i = 0$ then $\mathbf{c} = \mathbf{a}\sigma_{n-1}$, and the corresponding block in $\text{binary}(\otimes(P))$ is $001^{n-2} = 001^{n-3}\overline{b_i}$ by identity 1 in Remark 3. If $b_i = 1$ then $\mathbf{c} = \mathbf{a}\sigma_{n-2}$, and the corresponding block in $\text{binary}(\otimes(P))$ is $001^{n-3}0 = 001^{n-3}\overline{b_i}$ by identity 2 in Remark 3. $\square$

Recycling leads to binary representations that have only two distinct blocks: 001^{n-2} and $001^{n-3}0$ by (10). Further ‘re-recycling’ would lead to two distinct ‘mega-blocks’: $001^{n-1} \ 001^{n-1} \ 001^{n-2}0 \ \dots \ 001^{n-2}0 \ 001^{n-2}b$ for $b \in \{0, 1\}$. To utilize this observation, Theorem 16 proves that $D(n)$ is the SP-cycle obtained by continual recycling. To illustrate this fact, consider $D(3) = 321312$ (see [13]) and $D(4) = 432142134132431241234231$ from Fig. 1(d). Notice that the permutations of $D(3)$ are 321, 213, 132, 312, 123, 231 and recycling these permutations gives $D(4)$.

Theorem 16 *If the permutations of $D(n-1)$ are P , then $D(n) = \otimes(P)$.*

Proof If $\text{binary}(D(n-1)) = b_1 \cdots b_{(n-1)!}$, then equation (1) in [13] implies

$$\text{binary}(D(n)) = 001^{n-3} \overline{b_1} 001^{n-3} \overline{b_2} \cdots 001^{n-3} \overline{b_{(n-1)!}}. \quad (11)$$

Notice that (11) is identical to (10). The base case is $\text{binary}(D(2)) = 00$, which is the only binary representation of an SP-cycle for $\Pi(2)$. $\square$

In [13] a loopless algorithm similar to BinaryBell was given for generating successive bits of $\text{binary}(D(n))$. By Theorem 16 and Lemma 5, we can modify this algorithm to generate successive blocks of $\text{binary}(D(n))$. This is done by replacing lines 6–12 of BinaryBell with the following

$b \leftarrow [(j \text{ even}) \oplus (a_j - d_j \leq 0 \text{ or } a_j - d_j \geq n - 1 - j) \oplus (j = n - 1)]$
 output($001^{n-3}b$)

The expression $[(j \text{ even}) \oplus (a_j - d_j \leq 0 \text{ or } a_j - d_j \geq n - 1 - j)]$ gives the bit that was output in [13] (with $n - 1$ replacing n) where $\oplus$ is exclusive-or. This bit equals the value of b except on BinaryBell’s final iteration¹ when $j = n - 1$. Similar modifications allow for the output of ‘mega-blocks’ or ‘mega-mega-blocks’.

7 Ranking

In this section we show how to efficiently rank permutations in 7-order, as well as the permutations obtained by decoding the bell-ringer SP-cycle.

7.1 Ranking 7-order

Let $R_7(\mathbf{a})$ denote the rank of the permutation $\mathbf{a} = a_1 a_2 \cdots a_n \in \Pi(n)$ in 7_n . If $a_k = n$, then directly from Definition 1 we have

$$R_7(a_1 a_2 \cdots a_{k-1} n a_{k+1} \cdots a_n) = \begin{cases} 0 & \text{if } n = 1, \\ 0 & + n \cdot R_7(a_2 \cdots a_n) & \text{if } k = 1, \\ (n-k+1) + n \cdot R_7(a_1 \cdots a_{k-1} a_{k+1} \cdots a_n) & \text{if } k > 1. \end{cases}$$

Noting that the recursive part of the recurrence relation removes n in both cases, we can iterate to obtain

$$R_7(\mathbf{a}) = \sum_{j=0}^{n-1} r_{n-j} \cdot (n)_j,$$

where $(n)_j = n(n-1) \cdots (n-j+1) = n!/(n-j)!$ is the “falling factorial”. The value of r_{n-j} is 0 or is $n-j-k_{n-j}+1$ where k_{n-j} is the position of $n-j$ in

¹The algorithm in [13] complemented the final bit of $\text{binary}(D(n))$ by initializing $f_n f_{n-1} \cdots f_1 \leftarrow n+1 \ n-1 \cdots 1$; the initialization $d_n d_{n-1} \cdots d_1 \leftarrow 11 \cdots 1$ should also have been $d_{n+1} d_n \cdots d_1 \leftarrow 11 \cdots 1$.

the permutation obtained from $\mathbf{a}$ by deleting the elements $n - j + 1, \dots, n - 1, n$. In other words, $k_i = 1 + |\{\ell < a_i^{-1} : a_\ell < i\}|$. For $0 \leq j < n$ we then have

$$r_{n-j} = \begin{cases} 0 & \text{if } k_{n-j} = 1, \\ n - j - k_{n-j} + 1 & \text{if } k_{n-j} > 1. \end{cases}$$

The sequence of values k_i for $i = 1, 2, \dots, n$ is closely related to the number of inversions in $\mathbf{a}$ and can be efficiently computed using various algorithms; for example by successively inserting $a_1, a_2, \dots, a_n$ into an augmented red-black tree as described in [4] (see Chap. 14, on Augmenting Data Structures), and computing the “rank” of each successive element immediately after its insertion. Thus we can compute the rank using $O(n \log n)$ arithmetic operations (on numbers that can be as large as $n!$).

7.2 Ranking $B(n)$

We can use R_7 from Sect. 7.1 to efficiently rank the bell-ringer SP-cycle. This ranking can be done with a single call to R_7 , a constant number of additional arithmetic operations, and $O(n)$ comparisons and assignments.

Given $\mathbf{a} \in \Pi(n)$, let $R(\mathbf{a})$ denote the rank of the permutation $\mathbf{a}$ in $B(n)$, or equivalently the rank of its shorthand substring $a_1 \cdots a_{n-1}$. We assume $R(n \ n-1 \ \cdots \ 2 \ 1) = R(n \ n-1 \ \cdots \ 2) = 0$ since SP-cycles ‘start’ with the substring $n \ n-1 \ \cdots \ 2$. Our computation requires the consideration of several cases.

Suppose that we have been given one of $B(n)$ ’s substrings $\mathbf{s}$. If $n \notin \mathbf{s}$ (so that $\mathbf{s} \in \Pi(n-1)$), then $R(\mathbf{s}) = 1 + n \cdot R_7(\mathbf{s})$. Otherwise, the substring has the form

$$\mathbf{s} = a_{j+1} \cdots a_{n-1} \ n \ b_1 \cdots b_{j-1}, \quad (12)$$

where $\mathbf{a}$ and $\mathbf{b}$ are two successive permutations in γ_{n-1} . The key to computing R is to determine $\mathbf{a}$. This is because $R(\mathbf{a}) = n \cdot R_7(\mathbf{a})$, and so $R(\mathbf{s}) = 1 + j + R(\mathbf{a})$ where j is defined by (12). We consider three cases. In these cases we let m be the symbol from $\langle n \rangle$ that is ‘missing’ from $\mathbf{s}$.

Case A: $n-1 \notin \{a_{j+1}, \dots, a_{n-1}\} \cup \{b_1, \dots, b_{j-1}\}$. Here it follows that $n-1 = a_1 = b_{n-1}$ and thus

$$\mathbf{a} = n-1 \ b_1 \cdots b_{j-1} \ a_{j+1} \cdots a_{n-1}.$$

Case B: $n-1 \in \{a_{j+1}, \dots, a_{n-1}\}$. Here we can conclude that

$$\mathbf{a} = b_1 \cdots b_{j-1} \ m \ a_{j+1} \cdots a_{n-1}.$$

Case C: $n-1 \in \{b_1, \dots, b_{j-1}\}$. We consider two subcases.

Subcase C1: $b_1 \neq n-1$. Suppose $b_h = n-1$. Then

$$\mathbf{a} = b_1 \cdots b_{h-1} \ b_{h+1} \ n-1 \ b_{h+2} \cdots b_{j-1} \ m \ a_{j+1} \cdots a_{n-1}.$$

Subcase C2: $b_1 = n-1$. This is the most complicated (and interesting) case. The permutation $\mathbf{b}$ will have the form

$$\mathbf{b} = n-1 \ n-2 \ \cdots \ d \ x_1 \ x_2 \ \cdots \ x_t \ d-1 \ y \ \mathbf{z}, \text{ or}$$

$$\mathbf{b} = n-1 \ n-2 \ \cdots \ d \ x_1 \ x_2 \ \cdots \ x_t \ d-1,$$

where $t > 0$, and in the first form y is a symbol and $\mathbf{z}$ is a string. Then

$$\mathbf{a} = x_1 \ n-1 \ n-2 \ \cdots \ d \ x_2 \ \cdots \ x_t \ y \ d-1 \ \mathbf{z}, \text{ or}$$

$$\mathbf{a} = d-1 \ n-1 \ n-2 \ \cdots \ d \ x_1 \ x_2 \ \cdots \ x_t, \text{ respectively.}$$

It should be evident that we can determine $\mathbf{b}$ from $\mathbf{s}$ (in $O(n)$ time), but we omit here the somewhat messy details of the various subcases that need to be considered. These subcases depend on how d and j (from (12)) are related.

Conjecture 1 has recently been proved in “Hamilton Cycles in Restricted Rotator Graphs” by Stevens and Williams, International Workshop on Combinatorial Algorithms, IWOCA 2011, LNCS, to appear.

References

1. Chung, F., Diaconis, P., Graham, R.: Universal cycles for combinatorial structures. *Discrete Math.* **110**, 43–59 (1992)
2. Compton, R.C., Williamson, S.G.: Doubly adjacent Gray codes for the symmetric group. *Linear Multilinear Algebra* **35**(3), 237–293 (1993)
3. Corbett, P.F.: Rotator graphs: an efficient topology for point-to-point multiprocessor networks. *IEEE Trans. Parallel Distrib. Syst.* **3**, 622–626 (1992)
4. Corman, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: *Introduction to Algorithms*, 3rd edn. MIT Press, Cambridge (2009)
5. Holroyd, A., Ruskey, F., Williams, A.: Faster generation of shorthand universal cycles for permutations. In: *Proceedings of the 16th Annual International Computing and Combinatorics Conference (COCOON 2010)*, Nha Trang, Vietnam, July 19–21. LNCS, vol. 6196, pp. 298–307. Springer, Berlin (2010)
6. Jackson, B.: Universal cycles of k -subsets and k -permutations. *Discrete Math.* **149**, 123–129 (1996)
7. Johnson, J.R.: Universal cycles for permutations. *Discrete Math.* **309**, 5264–5270 (2009)
8. Johnson, S.M.: Generation of permutations by adjacent transpositions. *Math. Comput.* **17**, 282–285 (1963)
9. Knuth, D.E.: *The Art of Computer Programming*, vol. 4, *Generating All Tuples and Permutations*, Fascicle 2. Addison-Wesley, Reading (2005)
10. Knuth, D.E.: *The Art of Computer Programming*, vols. 1–4A Boxed Set, 3rd edn. Addison-Wesley Professional, Reading (2011)
11. MacGregor, J.N., Ormerod, T.: Human performance on the traveling salesman problem. *Percept. Psychophys.* **58**(4), 527–539 (1996)
12. Page, J., Salvia, J., Collewet, C., Foresta, J.: Optimised De Bruijn patterns for one-shot shape acquisition. *Image Vis. Comput.* **23**, 707–720 (2005)
13. Ruskey, F., Williams, A.: An explicit universal cycle for the $(n-1)$ -permutations of an n -set. *ACM Trans. Algorithms* **6**(3), article 45 (2010)
14. Scheinerman, E.R.: Determining planar location via complement-free de Bruijn sequences using discrete optical sensors. *IEEE Trans. Robot. Autom.* **17**(6), 883–889 (2001)
15. Sedgewick, R.: Permutation generation methods. *Comput. Surv.* **9**, 137–164 (1977)
16. Sohn, H.S., Bricker, D.L., Simon, J.R., Hsieh, Y.C.: Optimal sequences of trials for balancing practice and repetition effects. *Behav. Res. Methods Instrum. Comput.* **29**, 574–581 (1997)
17. Williams, A.: Loopless generation of multiset permutations using a constant number of variables by prefix shifts. In: *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2009*, New York, NY, USA, January 4–6, pp. 987–996 (2009)
18. Williams, A.: *Shift gray codes*, PhD Thesis, University of Victoria (2009)